



---

**TECHNICAL UNIVERSITY**

OF CLUJ-NAPOCA, ROMANIA

**FACULTY OF AUTOMATION AND COMPUTER SCIENCE**  
**COMPUTER SCIENCE DEPARTMENT**

# StoryTeller

LICENSE THESIS

Graduate:

**Alexandra POP**

Supervisor:

**Prof. dr. ing. Dorian GORGAN**

**2026**



---

**TECHNICAL UNIVERSITY**  
OF CLUJ-NAPOCA, ROMANIA  
**FACULTY OF AUTOMATION AND COMPUTER SCIENCE**  
**COMPUTER SCIENCE DEPARTMENT**

---

DEAN,  
**Prof. dr. eng. Vlad MUREȘAN**

HEAD OF DEPARTMENT,  
**Prof. dr. eng. Rodica POTOLEA**

Graduate: **Alexandra POP**

## **StoryTeller**

1. **Project proposal:** Design and implementation of an autonomous and scalable software framework that decouples pedagogical narrative content from rigid graphical resources. The system integrates a Unity-based 3D interactive frontend with a Python-driven artificial intelligence backend to enable real-time, AI-based generation and dynamic adaptation of three-dimensional virtual learning environments, supported by an integrated Learning Management System (LMS) backend for real-time progress tracking, RBAC authentication, and automated student telemetry.
2. **Project contents:** Title page, project proposal and authenticity declaration, table of contents, Introduction, Project Objectives, Bibliographic Research, Analysis and Theoretical Foundation, Detailed Design and Implementation, Testing and Validation, User's Manual, Conclusions, Bibliography, Appendix (relevant code sections).
3. **Place of documentation:** Technical University of Cluj-Napoca, Computer Science Department
4. **Consultants:**
5. **Date of issue of the proposal:** October 1, 2025
6. **Date of delivery:** September 6, 2026

Graduate:

Supervisor:

**TECHNICAL UNIVERSITY**

OF CLUJ-NAPOCA, ROMANIA

**FACULTY OF AUTOMATION AND COMPUTER SCIENCE****COMPUTER SCIENCE DEPARTMENT****Declarație pe propria răspundere privind  
autenticitatea lucrării de licență**Subsemnatul(a) **Pop****Alexandra**

\_\_\_\_\_,  
legitimată(ă) cu C.I. \_\_\_\_\_ seria \_CJ\_\_\_\_\_ nr. \_593926\_\_\_\_\_  
(conform copiei anexate prezentei declarații, copie semnată și certificată "conform cu  
originalul"), autorul lucrării  
\_\_\_\_\_  
StoryTeller\_\_\_\_\_

\_\_\_\_\_  
elaborată în vederea susținerii examenului de finalizare a studiilor de licență la  
Facultatea de Automatică și Calculatoare, Programul de studii  
Calculatoare

din cadrul Universității Tehnice din Cluj-Napoca, sesiunea \_\_\_\_\_septembrie\_\_\_\_\_ a  
anului universitar \_\_\_\_\_2026\_\_\_\_\_, declar pe propria răspundere că această  
lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza  
informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite  
cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de  
examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative,  
respectiv, anularea examenului de licență.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea  
autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine și  
conform copiei documentului de identitate menționat mai sus.

Data

\_\_\_\_\_  
06.09.2026\_\_\_\_\_

Nume, Prenume

\_\_\_\_\_  
POP Alexandra\_\_\_\_\_

Semnătura

## Table of contents

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Chapter 1. Introduction .....                                         | 5  |
| Chapter 2. Project Objectives .....                                   | 8  |
| 2.1.    Project Theme .....                                           | 8  |
| 2.2.    Project Objectives.....                                       | 9  |
| Chapter 3. Bibliographic Research .....                               | 11 |
| 3.1 Architectural Frameworks of Modern Game Engines .....             | 11 |
| 3.1.1. Temporal Dynamics and Real-Time Simulation Paradigms.....      | 11 |
| 3.1.2. Game Loop Architectures and Multiprocessor Concurrency.....    | 12 |
| 3.1.3. Resource Management and Asset Conditioning Pipelines.....      | 12 |
| 3.2 Recurrent and Self-Attention Sequence Modeling .....              | 13 |
| 3.2.1 Architectural Bottlenecks of Recurrent Neural Networks .....    | 13 |
| 3.2.2 The Self-Attention Paradigm and the Transformer Solution.....   | 13 |
| 3.2.3 Complexity Analysis and Practical Efficiency Trade-offs .....   | 14 |
| 3.2.4 Convolutional and Restricted Field Alternatives.....            | 15 |
| 3.2.5 In-Context Learning and the Meta-Learning Paradigm.....         | 15 |
| 3.3 Evolution of Generative Audio Architectures .....                 | 15 |
| 3.3.1 Concatenative and Parametric Speech Generation .....            | 16 |
| 3.3.2 The Generative Audio Paradigm: Raw Waveform Modeling .....      | 16 |
| 3.3.3 Real-Time Audio Challenges.....                                 | 17 |
| 3.4 Integration of GenAI in Educational Frameworks.....               | 17 |
| 3.4.1 Personalized Learning and Adaptive Pedagogical Scaffolding..... | 17 |
| 3.4.2 Enhancing Student Engagement and Cognitive Development .....    | 18 |
| 3.4.3 Challenges and Risks in AI-Assisted Learning .....              | 18 |
| 3.4.4 Balancing Innovation with Responsible Design .....              | 20 |
| 3.5 Frameworks for AI Educational Impact .....                        | 20 |
| 3.5.1 Constructivist Learning Models.....                             | 20 |
| 3.5.2 Learner Autonomy and Self-Regulated Learning.....               | 21 |
| 3.5.3 The Necessity of AI Literacy and Assessment Redesign .....      | 21 |
| 3.5.4 Cognitive Multimedia Learning Theory .....                      | 21 |
| Chapter 4. Analysis and Theoretical Foundation.....                   | 23 |
| 4.1. Conceptual Architecture and System Overview .....                | 23 |
| 4.2. Functional Structure and Use Case Modeling .....                 | 24 |

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| 4.3. Learning Management Logic and Abstract Data Models .....               | 25 |
| 4.4. Prompt Engineering Logic and AI Constraint Mechanisms .....            | 26 |
| 4.5. Finite State Machine (FSM) Logic of the Interactive Client .....       | 27 |
| 4.6. Abstract Entity Relationship and Data Propagation .....                | 28 |
| 4.7. Adaptive Scoring Algorithm Logic .....                                 | 29 |
| 4.8. Threat Modeling and AI Prompt Injection Mitigation.....                | 29 |
| 4.9. Spatial Collision and Event-Driven Logic.....                          | 30 |
| 4.10. Sequence Modeling of the Generative Interaction.....                  | 31 |
| 4.11. System Specifications and Constraints .....                           | 32 |
| 4.11.1. Functional Specifications.....                                      | 32 |
| 4.11.2. Non-Functional Specifications.....                                  | 33 |
| Chapter 5. Detailed Design and Implementation .....                         | 34 |
| 5.1. General Application Architecture and Technology Stack .....            | 34 |
| 5.2. Database Design and Data Persistence .....                             | 35 |
| 5.3. Backend Generative Module (Python Implementation) .....                | 36 |
| 5.3.1. Prompt Engineering and JSON Schema Enforcement.....                  | 36 |
| 5.3.2. Sequential Image Generation Guarantee .....                          | 37 |
| 5.4. Frontend Orchestration and IPC Implementation (GameInterface.cs) ..... | 38 |
| 5.5. Central Orchestrator and LMS Controller (StoryDirector.cs) .....       | 38 |
| 5.5.1. Sequence Execution and Asset Coordination .....                      | 39 |
| 5.5.2. Component Configuration (StoryDirector).....                         | 39 |
| 5.5.3. Authentication and Role-Based Login Interface.....                   | 40 |
| 5.5.4. Educator Dashboard and Class Student Monitoring Interface.....       | 41 |
| 5.5.5. Teacher Generation Interface and Parameter Configuration .....       | 42 |
| 5.6. Deterministic Physics and Camera Control (ActorController.cs).....     | 43 |
| 5.7. Dynamic Assessment Module (QuizManager.cs).....                        | 45 |
| 5.8. Audio Synthesis, Streaming, and Subtitle Interleaving .....            | 46 |
| 5.8.1. Server-Side Audio Generation (genereaza_audio) .....                 | 47 |
| 5.8.2.....                                                                  | 47 |
| 5.9. Fault Tolerance and Graceful Degradation (AI_FAILED Architecture)...   | 47 |
| 5.9.1. Backend Exception Handling and Error Flagging .....                  | 47 |
| 5.9.2. Frontend Error Detection (LoadAndRunStory) .....                     | 48 |
| 5.10. Asset Lifecycle and Cache Management .....                            | 48 |
| 5.10.1. Cache Purging (CurataImaginiVechi).....                             | 48 |
| 5.10.2. Teacher Repository Archiving .....                                  | 48 |
| 5.11. Advanced Telemetric Tracking and Prerequisite Auto-Repair.....        | 49 |
| 5.11.1. Prerequisite Auto-Repair Algorithm (Elev_IncarcaLectii).....        | 49 |

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Chapter 6. Testing and Validation .....                                      | 49 |
| 6.1. Testing Strategy and Objectives .....                                   | 50 |
| 6.2. Component-Level Unit and Functional Testing .....                       | 50 |
| 6.2.1. Database Initialization and RBAC Validation.....                      | 50 |
| 6.2.2. Deterministic Movement Validation.....                                | 51 |
| 6.3. Integration Testing of the Inter-Process Communication (IPC) Pipeline.. | 51 |
| 6.3.1. Process Spawning and Stream Redirection.....                          | 51 |
| 6.4. Generative Performance and Latency Benchmarking .....                   | 51 |
| 6.4.1. Component-Level Latency Breakdown .....                               | 52 |
| 6.5. Fault Tolerance and Error Recovery Validation .....                     | 52 |
| 6.5.1. JSON Schema Enforcement and Cleaning Validation.....                  | 53 |
| 6.5.2. Image Download Resilience (Pollinations AI) .....                     | 53 |
| 6.5.3. Graceful Degradation (AI_FAILED Resilience) .....                     | 54 |
| 6.6. Pilot Usability and Demographic Evaluation .....                        | 54 |
| 6.6.1. Participant Profile and Methodology .....                             | 54 |
| 6.6.2. Key Findings and Observations .....                                   | 54 |
| 6.7. Storage Scalability and Resource Utilization .....                      | 54 |
| 6.8. Summary Of Testing.....                                                 | 55 |
| Chapter 7. User's Manual .....                                               | 56 |
| 7.1. Installation and Deployment Procedure (Administrator's Guide).....      | 56 |
| 7.1.1. Hardware and Software Prerequisites.....                              | 56 |
| 7.1.2. Step-by-Step Deployment Instructions .....                            | 56 |
| 7.2. End-User Operation Manual .....                                         | 57 |
| 7.2.1. Authentication and Role-Based Access Control (RBAC) .....             | 57 |
| 7.2.2. Teacher Workflow: Lesson Generation and Publishing .....              | 57 |
| 7.2.3. Student Workflow: Exploration and Assessment .....                    | 58 |
| Chapter 8. Conclusions .....                                                 | 59 |
| 8.1. Summary of Contributions and Achievements .....                         | 59 |
| 8.2. Critical Analysis of Results Achieved .....                             | 59 |
| 8.3. Possibilities for Improvements and Further Development .....            | 60 |
| Bibliography .....                                                           | 61 |
| Appendix 1.....                                                              | 62 |
| A.1. Asynchronous IPC and WSL Process Orchestration .....                    | 62 |
| A.2. Student Progress Auto-Repair and State Enforcement .....                | 63 |
| A.3. Backend JSON Schema Cleaning and Deserialization Fallback .....         | 64 |

## List of Figures:

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Figure 4.1: High-level conceptual architecture of the StoryTeller system ..... | 23 |
| Figure 4.2: UML Use Case diagram illustrating system interactions.....         | 25 |
| Figure 4.3: Educational Client State Transition Diagram.....                   | 28 |
| Figure 4.4: UML Sequence Diagram of the Generative Interaction.....            | 32 |
| Figure 5.1: Entity-Relationship Diagram of Database Schema .....               | 35 |
| Figure 5.2: Document-Oriented Database Schema Architecture.....                | 36 |
| Figure 5.3: Role-Based Access Control Workflow Architecture .....              | 39 |
| Figure 5.4: StoryDirector Inspector Configuration.....                         | 40 |
| Figure 5.5: Authentication and User Login Interface .....                      | 41 |
| Figure 5.6: Educator Dashboard for Student Records .....                       | 42 |
| Figure 5.7: Custom Lesson Generation Dashboard .....                           | 43 |
| Figure 5.8: Physics Update Loop and Vector Normalization.....                  | 44 |
| Figure 5.9: Dynamic Assessment and Telemetry Feedback Flow.....                | 46 |

## List of Tables:

|                                                                        |    |
|------------------------------------------------------------------------|----|
| Table 3.1: Comparison of Layer Types and Complexity.....               | 14 |
| Table 4.1: Prompt Payload Parameters and Pedagogical Constraints ..... | 26 |
| Table 4.2: Data Entities and Lifecycle Breakdown .....                 | 29 |
| Table 4.3: Generative Content Threat Modeling .....                    | 30 |
| Table 6.1: Generative Pipeline Latency Breakdown .....                 | 52 |
| Table 6.2: Architectural Fault-Tolerance Scenarios .....               | 53 |

## **Chapter 1. Introduction**

Modern education is undergoing continuous development. As technology becomes part of our lives and younger generations become more and more familiar with it, educators face major obstacles in maintaining effective instructional practices. In terms of teaching, professors and educators are having trouble keeping children's attention, so they give up on static materials with long texts and bidimensional images and resort to familiar examples and associations, e-learning platforms, and video overviews of the subject to help with information retention and intellectual involvement. The monotony of classic materials is in complete contradiction to the widely colorful stimuli they encounter every day on their devices, and this discontinuity creates barriers to successful learning.

Immersive learning comes as a solution to standard teaching methods by placing learners in interactive, realistic instances in which they transition from passive consumption of information to making decisions and receiving instant feedback in an active, “hands-on” manner. This procedure matches how the human brain processes and remembers information. Activating multiple senses and emotional responses creates long-term memory anchors, helping newly learned information become more strongly encoded in long-term memory. Although it has benefits, immersive learning comes at a high price when we think about including these contexts into mainstream education. Digital simulations require expensive, specialized hardware that is not available to a wide range of institutions. Building realistic, interactive 3D environments takes advanced technical skills that go well beyond a regular teacher's digital expertise. It also means developers have to put in a huge amount of manual work. Because of this, it is very hard to quickly create or adjust content to match a specific class curriculum. There is also a need for automated, adaptive techniques that can overcome the differences between 3D virtual environments and dynamic content generation.

Even if not through engaged learning, most school institutions move towards technology by introducing e-learning platforms. E-learning platforms rely on static media, videos, and quizzes. They use 2D interfaces such as PCs, tablets, or phones to make information available anywhere, at any time, allowing students to click through presentations, take quizzes, and see teacher updates in real time. In other words, the scope of digital integration in early childhood education is categorized into two distinct domains: interactive foundational learning and administrative management. Administrative platforms are becoming increasingly integrated into early childhood education to support parents and educators. These platforms grant real-time visibility into children's learning routines, including real-time notifications and daily itineraries. In this way, parents can see instant notifications about nap time, their sleep schedule, food intake, and nutritional values for each meal, as well as the children's overall mood and emotional state. Administrative platforms also provide progress evaluation regarding sports activities or foreign language acquisition, useful not only for the parents but also for the monthly evaluation of the child, done by the institution. Parents can also see a centralized financial archive, allowing them to review monthly payment histories and administrative records.

Even if extremely helpful, both e-learning platforms and system management platforms do not focus on the “hands-on” experience and cognitive involvement given by language. Language immersion is an educational approach where a second language is used as the medium of instruction for academic content for at least half of the school day. It functions as a non-digital model for this deep cognitive involvement. Unlike non-interactive learning, successful language immersion relies mostly on discovery and inquiry-based learning, where students

must use their logic to decipher context clues. To make academic content comprehensible in a new language, immersion teachers must employ a highly active, "hands-on" approach that utilizes actual objects- called "realia"- manipulatives, gestures, body language, and movement instead of relying just on direct explanation or static text. In fact, this immersive technique produces proven benefits for the human brain that are hard to replicate for standard educational models. Research shows that bilinguals and students in language immersion programs develop increased mental flexibility, divergent thinking, and enhanced pattern recognition and decision-making skills. These students also display greater selective attention, which allows them to decode problems with ambiguous or misleading cues more quickly than their monolingual colleagues. Additionally, this cognitive-based engagement does not take away from basic education. Studies show that English-proficient students in language immersion programs consistently perform as well as, or better than, non-immersion peers on standardized measures of reading and math.

Even if the cognitive and academic advantages of language immersion are well documented, translating this level of active engagement into the digital domain continues to be a crucial challenge. Throughout the world, the continuous transformation toward digital development in educational institutions and combined learning models needs a fundamental review of how we deliver pedagogical content. In early childhood and primary education, technology should no longer function as a supplementary feature. It must serve as a link connecting the stimulating, active environments children experience at home with structured institutional learning.

This project thesis is in accordance with the broader field of immersive technology and adaptive e-learning. It targets the intersection of 3D virtual environment design, software engineering, and artificial intelligence. The fundamental objective is to facilitate a necessary paradigm shift: moving away from the passive information typically displayed on flat 2D screens and, at the same time, avoiding the high cost of traditional VR simulations. Instead, the attention shifts toward developing a scalable, user-driven software framework that naturally promotes autonomous and engaged learning experiences.

Existing e-learning platforms are largely limited to 2D interfaces, based mostly on static presentations, videos, and basic quizzes. These tools fail to encourage brain activity and the cognitive participation needed to embed the basic information in the brain. At the same time, compared to experiential 3D and VR techniques, e-learning solves the financial problem of costly hardware equipment or development but fails to provide hands-on immersion. As a consequence, the gap has extended because no automated, fast, and publicly available tool can customize or generate a 3D scenario that is personalized to the specific curriculum or adaptable to the comprehension level of the students while also having administrative management for grading and comparing the progress of the individual student, as well as the class evolution. This is because the vast majority of digital tools designed specifically for early and intermediate education stagnate at administrative tasks that completely neglect the learning concept.

To close this gap, a fundamental shift is necessary: the educational narrative must be separated from the graphical assets. There is a multitude of educational videos available on the internet, but they are made starting from general ideas, not personalized or relevant to the exact situations experienced by the children in a specific class. To avoid waiting for the developers to create increasingly specific scenarios, the ideal solution would be to create an AI-driven architecture that is capable of generating and adapting 3D scenarios rapidly, based strictly on immediate need. This system would transform static lessons into adjustable storytelling experiences without any constraint or technical intervention. On top of that, making this project work on standard devices like phones, tablets, or PCs increases availability and discards the

need for expensive hardware equipment while retaining the cognitive benefits of personalized scenarios.

To handle these technical and availability barriers, this thesis suggests a pedagogical framework that combines 3D virtual spaces with dynamic content generation. The project combines software engineering, immersive environments, and artificial intelligence. Its scope is to turn static studying into an automated, highly personalized, and interactive storytelling experience.

## Chapter 2. Project Objectives

### 2.1. Project Theme

The current paradigm of standard education is based on the difference between the dynamic digital experiences with high interaction that children experience every day and the static and passive nature of the standard educational materials. This thesis proposes an innovative software framework: a digital application for storytelling and an immersive learning engine. The main theme of this project is the development of an autonomous architecture based on artificial intelligence that is capable of transforming educational narratives into interactive, three-dimensional environments, all on an accessible hardware platform.

The core of this system concentrates on its capacity to decouple the educational narrative from the graphical assets. Unlike existing educational software, which is based on predefined, rigid, and time-consuming 3D simulations, this project introduces an abstraction engine. This engine takes the pedagogical requirements of a lesson and dynamically orchestrates the generation of the virtual scene. By using artificial intelligence techniques to generate the procedural content and the adaptive response logic, this system ensures that the created environment remains receptive at the individual needs and the level of understanding of each class.

This approach has two main objectives. First, it eliminates the technical and financial barriers of immersive learning. It does this by replacing the configuration complexity of the VR - conditioned by specialized hardware, with a framework focused on software, optimized for artificial intelligence. In this way, the application becomes accessible for a large number of educational institutions, regardless of their financial resources. Secondly, it shifts the educator's attention from the technical management and maintenance of the digital equipment to the creative facilitation of the learning process. This application acts as a bridge, allowing educators to use complex, immersive storytelling modules without requiring advanced technical abilities or endless hours of 3D manual development.

Additionally, this project integrates a robust administrative module that complements the experimental learning component. While the interactive environment concentrates on the cognitive implications and the memory anchor creation, the administration back-end offers live visibility of the child's progress. By allowing the educator to see the real-time progress of the students guarantees an all-inclusive view of the educational process, offering parents and educators a clear view of the learning process and levels of new language and emotional engagement metrics.

The fundamental cornerstone of this thesis is the focus put on pedagogical adaptability and continuous evaluation. Traditional e-learning platforms often treat the children as a uniform group, presenting the same content and static quizzes in the same manner. Instead, this project enhances the flexibility of the AI-based model to modify the scenario in real time and personalize it based on the class level. Also, to consolidate information retention, the system integrates dynamic quizzes at the end of each lesson, moving beyond standard rigid multiple-choice quizzes. If the educator detects a lack of implication, understanding, or performance during these interactive evaluations, they can create another lesson with a different difficulty level for the specific student or give personalized feedback. This ensures that the system is not

just an instrument for delivering information but also an intelligent assistant in the learning process with respect to the learning pace of each student.

Ultimately, this project is not just a standard e-learning project, but also an autonomous storytelling engine that promotes active learning based on analysis and understanding. By enabling the transition between passive assimilation of information and practical interaction, the framework proposes to reproduce the pedagogical benefits of immersive learning in a digital, scalable, and extremely personalized environment. By allowing the software environments to evolve in real-time to match the curiosity and cognitive development of the student, this framework creates a new standard for early education, shifting from a static consumption of content to a future defined by interactive, adaptive storytelling experiences.

## **2.2. Project Objectives**

This thesis requires a clear technological framework structured into a main objective and multiple thorough secondary ones. These objectives dictate the software design, the integration of artificial intelligence, and the pedagogical validation of the system.

The main objective:

Research, design, and implementation of an autonomous and scalable software architecture that successfully decouples the educational narrative content from rigid graphical resources. This central objective aims to enable the real-time, artificial intelligence-based generation and dynamic adaptation of three-dimensional virtual learning environments that can run smoothly on standard consumer hardware, thereby democratizing access to immersive educational tools.

Specific objectives:

Architectural design of the abstraction engine: Conceptualization and construction of a basic logical module that acts as an intermediary between the abstract pedagogical text and the graphical presentation. This engine must be able to translate the lesson requirements into structured environment parameters, without requiring manual intervention by the developer for each new scenario.

Implement AI-based procedural content generation: Integrate AI techniques capable of orchestrating and dynamically modifying 3D virtual scenes. The system must adapt the appearance, context, and elements of the virtual environment in real time, responding directly to the student's actions, choices, and learning pace.

Integrate dynamic and adaptive assessment mechanisms: To go beyond traditional, rigid, multiple-choice questionnaires by developing interactive assessment modules. These modules must assess information retention at the end of each narrative unit and autonomously feed performance data back into the system to adjust subsequent difficulty levels.

Develop a comprehensive administrative backend: To design a secure and centralized management and tracking interface. This component provides educators and parents with real-time analytics, monitoring language milestones, emotional engagement indicators, and historical progress archives to ensure a holistic assessment of the institution.

**System Validation and Accessibility Assurance:** To rigorously test and evaluate the framework's performance, resource consumption, and usability on standard devices (such as personal computers and tablets). This objective ensures that the system eliminates the financial barriers associated with specialized VR hardware while preserving the cognitive and psychological benefits of immersive learning.

## **Chapter 3. Bibliographic Research**

This bibliographic research chapter analyzes the foundational technological and pedagogical domains required to develop StoryTeller an interactive 3D educational storytelling platform. To construct this integrated architecture bridging a Unity-based frontend with a Python-driven artificial intelligence backend the literature review is structured across four essential pillars. Section 3.1 examines the architectural principles of modern game engines and real-time simulations. Section 3.2 explores the mathematical evolution of self-attention mechanisms and Large Language Models (LLMs) utilized for dynamic narrative and quiz generation. Section 3.3 investigates deep generative audio models required for natural text-to-speech synthesis. Finally, transitioning from hardware and algorithmic performance to human-computer interaction, Sections 3.4 and 3.5 analyze the pedagogical frameworks necessary for responsibly integrating generative AI into self-regulated learning environments. Together, these intersecting domains provide the theoretical justification for the proposed educational system.

### **3.1 Architectural Frameworks of Modern Game Engines**

In the domain of advanced interactive software engineering, video games are classified fundamentally as soft real-time, agent-based computer simulations [1]. Unlike hard real-time systems where failing to meet execution deadlines can lead to catastrophic hardware or safety failures, game engines operate under soft constraints [1]. In this environment, a missed deadline compromises only the user's subjective experience, introducing visual stuttering or input lag rather than system failure [1]. Designing a high-performance engine requires coordinating heterogeneous time-critical subsystems, structuring robust execution loops, and governing high-throughput asset pipelines [1].

#### **3.1.1. Temporal Dynamics and Real-Time Simulation Paradigms**

A modern game engine must satisfy complex temporal requirements across its core subsystems [1]. These requirements, referred to as deadlines, dictate the execution frequencies of various components [1]. For instance, rendering systems and display synchronization typically update at 30 Hz or 60 Hz to match monitor refresh rates and sustain the optical illusion of continuous motion [1]. Conversely, rigid-body dynamics and collision response solvers demand higher execution frequencies, such as 120 Hz, to preserve mathematical stability and prevent object interpenetration [1]. Meanwhile, artificial intelligence and high-level behavioral logic can execute at coarser intervals (once or twice per second) without harming user immersion [1], and low-level audio engines operate on strict sub-frame deadlines (such as every 1/60th of a second) to prevent buffer underruns [1].

To reconcile these conflicting temporal demands, game engines maintain distinct abstract timelines [1]. The Real Timeline tracks absolute physical "wall-clock" elapsed time using high-resolution hardware performance counters, utilizing 64-bit registers to prevent rapid integer wrapping [1]. In parallel, the Game Timeline governs the logical progression of the virtual simulation [1]. This logical timeline can be scaled, paused, or single-stepped independently of physical time [1]. Furthermore, floating-point precision limits dictate strict design choices; storing cumulative, long-running game time inside standard 32-bit floating-point variables is heavily discouraged because the IEEE-754 mantissa loses fractional precision as the clock's magnitude increases [1]. Engines mitigate this by restricting floats strictly to incremental time deltas and using integer clocks for global duration tracking [1].

Managing time deltas is critical to simulation stability [1]. Heavy frame-rate spikes can induce a vicious cycle where an engine attempts multiple physics integration steps to catch up, further overloading the CPU [1]. Similarly, developer debugging breakpoints require strict time-delta clamping (capping any frozen interval exceeding one second to a standard frame duration) to prevent physics explosions upon resuming execution [1].

### **3.1.2. Game Loop Architectures and Multiprocessor Concurrency**

At the core of any game engine lies the game loop, acting as the master execution driver that periodically services all subsystems [1]. Traditional single-threaded loops rely on platform-specific message queues such as Windows message pumps that service OS events before executing simulation updates or framework-driven callback models [1]. However, the modern proliferation of multi-core processors has forced architectures to evolve toward fine-grained concurrency [1].

Assigning a single dedicated thread to each major subsystem (task parallelism) is highly inefficient due to mismatched thread workloads, heavy synchronization stalls, and strict execution dependencies [1]. For instance, rendering and audio passes cannot execute until animation and physics calculations finish for that frame [1]. To overcome this, modern architectures utilize data-parallel scatter/gather patterns and job systems [1]. Frame workloads are broken down into granular, self-contained units of work known as jobs, which a centralized scheduler distributes dynamically across a pool of persistent worker threads pinned to specific physical CPU cores [1]. Advanced engines implement these jobs as coroutines or cooperative fibers, allowing a running task to yield control safely when encountering asynchronous bottlenecks, such as file streaming or ray-casting queries, thereby maximizing overall hardware utilization [1].

### **3.1.3. Resource Management and Asset Conditioning Pipelines**

Because video games are data-intensive multimedia systems, engines must govern the loading, parsing, and lifetime of heterogeneous assets including 3D meshes, textures, animations, and audio clips [1]. To ensure cross-platform compatibility, engines wrap native operating system file APIs inside a unified platform independence layer [1]. This layer introduces critical capabilities such as asynchronous background streaming, which allows assets to load dynamically during gameplay without triggering disruptive loading screens [1]. Streaming operations enforce strict priorities; real-time audio buffers, for example, take precedence over non-critical visual textures to prevent audible dropouts [1].

Because raw digital content creation (DCC) formats (such as Maya, Photoshop, or Houdini) contain redundant metadata and are structurally too complex to parse efficiently during runtime, engines employ an offline Asset Conditioning Pipeline (ACP) [1]. Through custom exporters, resource compilers, and linkers, raw assets are transformed into optimized, platform-specific binary packages managed by a central resource database [1]. At runtime, resource managers utilize Global Unique Identifiers (GUIDs), central lookup registries, and pointer fix-up tables to resolve dependencies and guarantee single-instance memory allocation [1]. Finally, to prevent heap memory fragmentation on hardware lacking advanced virtual memory page mapping, engines rely on deterministic allocators such as stack-based allocators for sequential, level-scoped lifecycles and chunk-based pool allocators for background asset streaming [1].

Aligning with these established design paradigms, the StoryTeller system adopts a modular resource and data-driven approach. Rather than relying on heavy proprietary binary packaging, the application utilizes a centralized JSON configuration file (`story.json`) stored within the Unity engine's `StreamingAssets` directory. This decouples dynamic runtime logic from external media assets (such as synthesized audio streams and procedural image textures), reflecting the

core principles of structured resource management and modular architecture outlined in contemporary game engine literature [1].

## **3.2. Recurrent and Self-Attention Sequence Modeling**

Historically, sequential transduction tasks like machine translation and language modeling relied on recurrent neural networks (RNNs) or convolutional neural networks (CNNs) structured within encoder-decoder frameworks [2]. While architectures incorporating Gated Recurrent Units (GRUs) or Long Short-Term Memory (LSTM) cells were established as the standard foundations, they present critical limitations in computational throughput and structural scalability [2].

### **3.2.1. Architectural Bottlenecks of Recurrent Neural Networks**

Standard sequential architectures process data by stepping through a sequence timeline from beginning to end [2]. At each step, they construct an internal representation, or hidden state, by combining the current input token with the historical representation computed in the preceding interval [2]. This temporal serialization creates an execution dependency where the state at any point cannot be evaluated until all prior iterations have finished executing [2]. Consequently, this structural layout blocks the division of computations across parallel processing units, establishing a major bottleneck when handling extended texts where hardware memory constraints limit batching efficiency [2].

Additionally, capturing connections between highly separated elements in a sequence is a persistent struggle for recurrent models [2]. The difficulty of training a network to model these distant interactions is heavily determined by how far forward and backward signals must travel across the computational graph [2]. In recurrent layers, the distance that a signal must bridge between any two arbitrary positions scales proportionally to the length of the sequence [2]. As a result of this linear distance constraint, mathematical gradients can easily degrade or dissipate when propagating across long temporal gaps, making the retrieval of global context highly unstable [2].

### **3.2.2. The Self-Attention Paradigm and the Transformer Solution**

The Transformer architecture addresses these limitations by substituting recurrence with a pure self-attention mechanism, which models relationships across a sequence without utilizing step-by-step state propagation [2]. Also referred to as intra-attention, this mechanism directly relates every position of a sequence to every other position to construct a unified contextual representation [2].

Mathematically, self-attention maps an array of query vectors against a set of key-value pairs to calculate a weighted output representation [2]. By packaging these queries, keys, and values into respective matrices, the normalized attention scores are computed simultaneously across all positions by multiplying the query matrix with the transposed key matrix, scaling the result by the square root of the key dimension, applying a softmax normalization function, and finally multiplying the resulting weight matrix by the value matrix [2].

This technical shift offers distinct structural advantages [2]:

- **Massive Parallelization:** Because this calculation executes across the entire sequence at once, a self-attention layer connects all positions using a constant number of sequential operations, requiring a complexity order of one [2]. This structural shift removes the step-by-step

sequential constraint of recurrent networks, allowing training to be massively parallelized on modern hardware [2].

- **Constant Signal Path Length:** The maximum path length that a signal must traverse between any two arbitrary positions in a self-attention network is reduced to a constant order of one [2]. This direct, single-step path length prevents gradient degradation over long sequences, enabling the model to learn long-range and global dependencies significantly more easily [2].

### 3.2.3. Complexity Analysis and Practical Efficiency Trade-offs

Analyzing the theoretical properties of these layers reveals distinct trade-offs in computational efficiency, parallelization capability, and signal path lengths [2]:

| Layer Type                         | Complexity per Layer                                                         | Sequential Operations              | Maximum Path Length                                          |
|------------------------------------|------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------------------|
| <b>Self-Attention</b>              | Quadratic relative to sequence length multiplied by representation dimension | Constant order of one              | Constant order of one                                        |
| <b>Recurrent</b>                   | Linear relative to sequence length multiplied by the squared dimension       | Linear relative to sequence length | Linear relative to sequence length                           |
| <b>Convolutional</b>               | Scaled by kernel size, sequence length, and squared dimension                | Constant order of one              | Logarithmic relative to sequence length based on kernel size |
| <b>Self-Attention (restricted)</b> | Scaled by neighborhood size, sequence length, and dimension                  | Constant order of one              | Proportional to sequence length divided by neighborhood size |

Table 3.1: Comparison of Layer Types and Complexity

In terms of operations per layer, a recurrent structure scales linearly with sequence length multiplied by the squared dimension, whereas a self-attention layer scales quadratically with sequence length multiplied by the dimension [2]. Consequently, self-attention layers exhibit faster execution speeds than recurrent layers when the sequence length is smaller than the hidden feature dimensionality [2]. In state-of-the-art language models, sentences are typically split using subword tokenization schemes (such as word-piece or byte-pair representations) [2]. These representations yield sequence lengths that are relatively short compared to highly dimensional representation spaces, allowing self-attention to optimize parallel hardware far more effectively [2].

### **3.2.4 Convolutional and Restricted Field Alternatives**

Prior efforts sought to avoid sequential computation by using convolutional structures as their foundational units, executing calculations in parallel across all sequence positions [2]. However, these convolutional designs introduce structural limits, such as sub-global receptive fields, distance-dependent operation growth (scaling linearly or logarithmically with distance), and stacking overhead required to bridge global context [2]. Furthermore, when processing exceptionally long inputs where sequence length heavily exceeds the representation dimension, the quadratic scaling of full self-attention can become a performance constraint [2]. To maintain efficiency, architectures can restrict the attention field to a localized window centered around each respective token [2]. This reduces per-layer operational costs to a linear scale relative to sequence length, though it reintroduces a distance-dependent path length constraint [2].

#### **Relevance to the Proposed Solution (StoryTeller):**

While foundational sequence transduction is governed by massive transformer-based architectures, practical educational applications integrate these capabilities via high-level API integrations. The StoryTeller system utilizes external Large Language Model (LLM) APIs powered by state-of-the-art self-attention mechanisms in its backend [2]. Through structured programmatic prompts, the system receives dynamically generated JSON data containing narrative structures and educational quizzes, which are subsequently parsed and rendered inside the Unity environment.

### **3.2.5 In-Context Learning and the Meta-Learning Paradigm**

The transition from task-specific architectures to task-agnostic pretrained models has fundamentally redefined natural language processing pipelines. [7] Historically, even when utilizing generalized pretraining, models required a final task-specific fine-tuning step, involving gradient updates on thousands of labeled examples, to achieve competitive performance on downstream applications.[7] However, scaling autoregressive language models to unprecedented sizes exemplified by the 175-billion parameter GPT-3 architecture enables a significant paradigm shift toward task-agnostic, zero-, one-, and few-shot transfer capabilities without any structural weight modifications or gradient updates.[7]

Under this framework, the language model operates effectively as a meta-learner, combining a slow, outer-loop learning process during pretraining with a rapid, "in-context" learning phase inside its context activations during inference.[7] By providing a few natural language demonstrations or prompt-based task specifications within the input activation window, the model configures its outputs dynamically to match the desired task. [7] This in-context capability is highly relevant for interactive applications like StoryTeller, as it allows the narrative engine to generate coherent, contextually adapted story paths and dynamic quizzes on the fly, relying on prompt engineering rather than expensive and rigid offline model fine-tuning.[7]

## **3.3 Evolution of Generative Audio Architectures**

In modern multimedia applications, intelligent tutoring systems, and interactive educational frameworks, auditory feedback plays a pivotal role alongside visual interfaces [3]. Delivering natural, high-fidelity speech synthesis dynamically from raw textual input has historically stood as one of the most complex challenges in digital signal processing and human-computer interaction [3]. Traditional speech generation frameworks frequently struggled to produce organic-sounding voices, resulting in robotic, flat, or mechanical artifacts that actively

diminished user engagement and cognitive retention [3]. Examining the technological shift from legacy synthesis methodologies to contemporary deep generative audio models establishes an essential theoretical foundation for integrating responsive voice pipelines into interactive software environments [3].

### **3.3.1 Concatenative and Parametric Speech Generation**

Before the widespread adoption of deep neural network architectures, automated speech generation was dominated by two conventional paradigms: concatenative synthesis and statistical parametric synthesis [3].

**Concatenative Synthesis and Corpus Limitations:** This methodology relies on expansive, professionally recorded databases containing human speech samples that are meticulously segmented into fundamental linguistic units, such as phonemes, diphones, or half-syllables [3]. During runtime execution, the system algorithms select and seamlessly stitch together these pre-recorded audio fragments to construct target words and sentences [3]. Although concatenative systems can achieve high local clarity within narrow, restricted domains, they exhibit severe structural inflexibility [3]. Any localized modification to vocal emotion, speaking tempo, or sentence prosody requires intricate digital signal processing manipulations [3]. These operations frequently introduce audible distortions, phase cancellations, and unnatural glitches at concatenation boundaries [3]. Furthermore, maintaining comprehensive speech corpora demands intensive manual labor, rigorous phonetic annotation, and substantial storage overhead [3].

**Statistical Parametric Synthesis and Acoustic Modeling:** To overcome the rigid storage requirements of concatenative corpora, parametric frameworks generate speech entirely from mathematical models [3]. Utilizing statistical tools such as Hidden Markov Models (HMMs) or recurrent architectures like Long Short-Term Memory (LSTM) networks, these systems predict sequences of acoustic parameters including fundamental frequency contours, spectral envelopes, and phonetic durations directly from input text features [3]. A specialized signal processor, known as a vocoder, subsequently translates these synthesized parameters into an audible audio waveform [3]. While parametric models offer exceptional computational efficiency and high flexibility for real-time voice modification, their primary drawback lies in subjective listening quality [3]. The resulting audio frequently exhibits a muffled, buzzy, or synthetic texture, driven by the over-smoothing characteristics of statistical parameter averaging and the inherent limitations of traditional vocoding algorithms [3].

### **3.3.2 The Generative Audio Paradigm: Raw Waveform Modeling**

To bridge the substantial quality gap separating synthetic speech from natural human articulation, modern machine learning introduced a fundamental paradigm shift by operating directly on raw audio waveforms [3]. Rather than depending on intermediate acoustic parameters or assembling fragmented pre-recorded clips, advanced generative audio models compute the conditional probability distribution of raw audio sample sequences directly across time [3].

By modeling acoustic signals at high temporal resolution frequencies typically ranging between 16 kHz and 24 kHz these deep architectures capture intricate, fine-grained physical details that traditional vocoders invariably discard [3]. Such details include micro-acoustics like vocal cord friction, breath intakes, lip movements, and environmental room reverberation [3]. When evaluated through rigorous subjective human evaluation frameworks, such as Mean Opinion

Scores (MOS), deep generative waveform models have proven capable of reducing the perceptual quality gap relative to natural human speech by over fifty percent across diverse linguistic contexts, including both inflection-heavy European languages and tonal languages like Mandarin Chinese [3].

### **3.3.3 Real-Time Audio Challenges**

Integrating high-quality speech synthesis into software applications introduces strict technical demands regarding resource allocation and real-time synchronization [3]. In interactive application architectures, auditory feedback must be tightly coupled with visual rendering updates [3]. Because sound propagation reinforces visual actions and directs user focus, the audio subsystem operates under uncompromising scheduling deadlines [3].

Failing to service audio output buffers within tight temporal windows such as every fraction of a millisecond dictated by hardware buffer sizes results in immediate buffer underruns, audible popping, or jarring latency delays [3]. Consequently, game engines must manage audio data through dedicated streaming channels that take priority over non-critical visual asset loading, ensuring continuous, uninterrupted delivery during active user interaction [3].

While the preceding sections established the computational, algorithmic, and architectural requirements for real-time AI integration, evaluating an educational platform requires more than hardware performance metrics. The technical capabilities of text and speech generation must be grounded in established qualitative methodologies to ensure genuine human benefit. Therefore, the following sections shift focus from algorithmic complexity to the educational impact of these technologies, examining how AI systems interact with cognitive development.

## **3.4 Integration of GenAI in Educational Frameworks**

In contemporary educational technology, the rapid advancement and accessibility of Generative Artificial Intelligence (GenAI) have initiated a profound digital transformation across learning environments [4]. Unlike traditional e-learning systems which rely on rigid, pre-programmed decision trees or static question-and-answer databases GenAI systems leverage deep neural networks to produce context-sensitive text, dynamic media, and adaptive instructional materials in real-time response to user prompts [4]. Exploring the theoretical benefits, pedagogical impacts, and structural challenges of GenAI integration provides crucial context for designing intelligent, interactive software systems tailored for active learning and knowledge acquisition [4].

### **3.4.1 Personalized Learning and Adaptive Pedagogical Scaffolding**

One of the most significant pedagogical advantages associated with generative artificial intelligence is its capacity for highly individualized, adaptive instruction [4]. Traditional classroom settings often struggle to accommodate the unique paces and learning curves of individual students, as educators must address the collective needs of a group [4]. GenAI models function as virtual tutoring assistants capable of delivering customized explanations, simplified analogies, and targeted examples tailored precisely to a learner's current comprehension level [4].

From the perspective of constructivist learning theory, knowledge is actively built by learners through interaction with their environment and responsive feedback [4]. Generative systems support this paradigm by offering continuous, non-judgmental interactions where students can repeat questions, request alternative perspectives, and explore complex topics at their own

individual pace [4]. Furthermore, when integrated into digital storytelling and problem-solving platforms, adaptive AI tools can dynamically modify content complexity based on user input, thereby sustaining student engagement and fostering higher levels of self-regulated learning [4].

The deployment of such large-scale conversational models in primary and secondary educational contexts establishes a highly customizable learning ecosystem.[8] Large language models can act as personalized pedagogical agents that stimulate student curiosity, enhance question-asking abilities, and provide step-by-step problem-solving guides across various curricula. [8] Furthermore, from the educator's perspective, these systems dramatically reduce the administrative overhead of manual lesson planning and resource design by automatically synthesizing tailored syllabi, practice problems, and distractors for multiple-choice assessments.[8] However, the successful integration of LLMs within classroom curricula is constrained by the inherent "brittleness" and lack of interpretability of probabilistic systems, which demands the proactive development of student AI literacy, rigorous fact-checking strategies, and persistent human oversight to mitigate cognitive over-reliance and algorithmic bias.[8]

### **3.4.2 Enhancing Student Engagement and Cognitive Development**

The incorporation of generative multimedia into educational applications has been shown to positively influence student motivation, curiosity, and task persistence [4]. Interactive learning modules that combine automated text generation with visual and auditory feedback encourage learners to move from passive consumers of information to active participants in the knowledge-creation process [4].

Studies examining cognitive outcomes indicate that GenAI-supported environments can improve narrative comprehension, vocabulary acquisition, and critical thinking skills [4]. By presenting multi-perspective arguments, posing open-ended exploratory questions, and providing immediate feedback loops, these systems encourage learners to analyze information critically rather than accepting static answers [4]. However, researchers emphasize that the realization of these cognitive benefits depends heavily on the presence of structured instructional design and guided mediation [4].

### **3.4.3 Challenges and Risks in AI-Assisted Learning**

Despite the extensive opportunities offered by generative technologies, their integration into educational frameworks introduces several notable challenges and ethical concerns that must be addressed by system designers and educators [4]:

**Accuracy and Information Hallucination:** GenAI models operate on probabilistic token prediction rather than deterministic fact retrieval [4]. Consequently, they are susceptible to generating plausible-sounding but factually incorrect information, commonly referred to as "hallucinations" [4]. In educational contexts, where learners may lack the domain expertise to independently verify complex outputs, exposure to inaccurate AI-generated content can lead to persistent misconceptions [4].

**Over-reliance and Critical Thinking Risks:** The sheer speed and convenience with which generative tools provide solutions can foster cognitive dependency [4]. If students rely excessively on AI systems to complete assignments or solve problems without engaging in

independent reflection, opportunities for developing higher-order analytical skills and genuine problem-solving capabilities may be compromised [4].

**Academic Integrity and Assessment Vulnerability:** The ability of conversational agents to generate sophisticated essays, code blocks, and structured responses has disrupted traditional evaluation methods [4]. Educational institutions face ongoing challenges regarding authorship, attribution, and the prevention of academic dishonesty, necessitating a shift toward project-based assessments, oral evaluations, and critical analysis tasks [4].

**The Digital Divide and Accessibility Inequities:** Access to advanced generative tools, high-speed internet infrastructure, and digital literacy training is unevenly distributed across socioeconomic groups and geographic regions [4]. This disparity creates a risk that AI-enhanced educational advantages will widen existing achievement gaps rather than promoting universal inclusion [4].

These accessibility inequities are further empirically validated by a comprehensive UK-based study conducted by The Alan Turing Institute, which investigated the domestic and institutional usage of generative systems among children aged 8 to 12. [6] The research highlights a pronounced socio-economic digital divide, demonstrating that household income, regional geographic distribution, and institutional school type (specifically, private versus state-funded education) heavily dictate a child's level of AI awareness, exposure, and access.[6] While parents and educators express general optimism regarding the creative and administrative utilities of generative tools, they simultaneously raise significant concerns regarding the erosion of children's critical thinking skills, exposure to unverified or inaccurate information, and potential compromises to academic integrity.[6]

Furthermore, the study's scenario analyses reveal that despite the rapid integration of GenAI in society, a vast majority of primary learners still fundamentally rely on traditional search engines and analog physical media for homework and creative tasks, with less than 10% proactively utilizing conversational agents.[6] This empirical baseline underscores the necessity of designing highly guided, intuitive interfaces like StoryTeller, which actively support cognitive development through structured, multimodal scaffolding while mitigating the structural risks and inequities associated with unsupervised AI exposure.[6]

The structural challenge of the digital divide is deeply intertwined with the broader timeline of technology adoption and infrastructure readiness in educational environments.[10] As highlighted in the NMC Horizon Project analysis, emerging technologies consistently drive a shift in student expectations, forcing academic systems to transition toward mobile-first content delivery and flexible, user-centered digital strategies.[10] The integration of adaptive, hands-on learning experiences such as digital makerspaces and online databases remains highly dependent on the institution's ability to address "wicked challenges," including managing knowledge obsolescence and embracing rapid organizational changes.[10]

Consequently, deploying a cross-platform, decoupled storytelling engine on accessible devices like standard tablets and smartphones directly addresses these historical accessibility barriers, aligning with the strategic push to integrate adaptive technologies directly into mainstream educational spaces.[10]

### 3.4.4 Balancing Innovation with Responsible Design

To maximize the efficacy of generative artificial intelligence in educational software, systems must be designed with a strong emphasis on pedagogical intent rather than mere technological novelty [4]. AI should act as a cognitive scaffolding tool supporting exploration, providing structured hints, and sparking creative inspiration rather than functioning as an autonomous replacement for human intellectual effort [4]. Establishing transparent system boundaries, enforcing strict content verification protocols, and promoting digital literacy are vital prerequisites for deploying responsible educational technologies [4].

Aligning with current research trends in adaptive educational technology, the StoryTeller application bridges the gap between automated content generation and structured interactive learning. By utilizing generative AI models to dynamically construct personalized narrative storylines and corresponding evaluation quizzes, the platform provides tailored educational content to users. To mitigate risks associated with accuracy and unguided dependency, the system embeds these AI-generated challenges within a controlled, predictable Unity interface, ensuring that computational automation serves as a structured, supportive complement to the user's active learning process.

## 3.5 Frameworks for AI Educational Impact

The integration of artificial intelligence into higher education and interactive software requires a strong foundation in established pedagogical theories to ensure that technological advancements translate into genuine educational benefits [5]. Recent empirical studies evaluating the impact of Generative AI on learning outcomes highlight the intersection of Constructivist Learning Theory and the Technology Acceptance Model (TAM) [5]. Understanding these theoretical frameworks provides essential guidelines for developing software that actively enhances human cognition, encourages critical reflection, and prevents the passive consumption of automated answers [5].

### 3.5.1 Constructivist Learning Models

According to Constructivist Learning Theory, learning is an active, ongoing process where individuals construct new knowledge through continuous interaction with resources, environments, and experiences [5]. Generative AI aligns perfectly with this paradigm by functioning as an interactive virtual assistant rather than a static, one-way repository of information [5]. By allowing users to ask questions iteratively, explore alternative viewpoints, and receive tailored explanations, AI tools foster active cognitive engagement and intellectual curiosity [5].

Furthermore, the adoption and effectiveness of these interactive tools are heavily predicted by the Technology Acceptance Model (TAM), a widely recognized sociological framework which states that user acceptance is primarily driven by two factors: perceived usefulness and perceived ease of use [5]. In the context of generative models, students perceive high usefulness because AI provides immediate academic support such as brainstorming ideas, improving language structure, and breaking down highly complex topics into digestible analogies [5]. Simultaneously, the perceived ease of use is driven by the natural, conversational interfaces of these models [5]. When educational interfaces are designed to be accessible and highly responsive, learner motivation increases significantly, transforming technological novelty into sustained academic productivity [5].

### **3.5.2 Learner Autonomy and Self-Regulated Learning**

The deployment of interactive AI systems has a profound and measurable impact on behavioral and cognitive learning outcomes [5]. Research indicates that AI-enriched environments boost student engagement and foster highly effective self-directed learning practices [5]. Traditional educational settings often suffer from bottlenecks, as students must wait for instructor availability to resolve misunderstandings [5]. Generative AI removes this barrier by providing asynchronous, around-the-clock support [5].

By offering immediate feedback and customized problem-solving scenarios, generative models grant learners greater autonomy over their educational journey [5]. This allows individuals from diverse educational backgrounds to progress at their own unique speed, rectifying misconceptions in real-time [5]. Consequently, when AI is utilized strictly as a cognitive aid supporting the thinking and research process rather than bypassing it students report higher levels of confidence, reduced academic anxiety, and improved conceptual clarity when navigating complex topics [5].

### **3.5.3 The Necessity of AI Literacy and Assessment Redesign**

Despite the widely recognized educational benefits, maximizing positive learning outcomes requires actively mitigating the risks of algorithmic dependency and the propagation of unverified information [5]. Educational institutions and software designers must prioritize the development of "AI Literacy" [5]. This concept encompasses much more than the basic technical ability to operate generative tools; it requires the critical capacity to verify AI-generated data, understand the boundaries of algorithmic competence, formulate effective prompts, and apply rigorous ethical standards to avoid plagiarism [5].

Additionally, because traditional text-based assessments (such as standard essays or multiple-choice questionnaires) are increasingly vulnerable to automated completion by AI, educational frameworks must transition toward redesigned evaluation methods [5]. Researchers advocate for dynamic, project-based evaluations, oral presentations, reflective assignments, and interactive practical demonstrations [5]. Shifting the focus from static answer generation to active analytical reflection ensures that higher-order thinking skills such as critical analysis, creativity, and problem-solving are preserved and cultivated in the AI era [5].

This chapter has established the comprehensive literature and state-of-the-art technologies that underpin modern intelligent multimedia applications. The architectural principles of real-time game engines ensure the computational performance and efficient resource management required for a seamless Unity-based 3D environment [1]. The mathematical evolution of self-attention mechanisms provides the structural backbone for the Python-driven advanced natural language understanding and dynamic narrative generation [2]. Concurrently, deep generative waveform modeling synthesizes immersive, high-fidelity human speech [3]. Beyond technical execution, contemporary pedagogical research confirms that when these generative technologies are integrated thoughtfully, they significantly enhance personalized learning, user engagement, and learner autonomy [4], [5]. Ultimately, the convergence of real-time 3D rendering, transformer-based sequence modeling, and constructivist learning theory forms the robust scientific framework upon which StoryTeller is designed, architected, and evaluated.

### **3.5.4 Cognitive Multimedia Learning Theory**

To ensure that the dynamic digital outputs of game engines and generative AI translate into genuine cognitive growth, the visual and auditory interface of the application must be designed

in accordance with the Cognitive Theory of Multimedia Learning (CTML).[9] Rooted in the multimedia principle, which states that individuals learn more deeply from a combination of words and pictures than from words alone, CTML posits that the human mind processes information through separate visual and auditory channels, each characterized by a strictly limited processing capacity. [9] Consequently, educational technology must reject machine-centered designs in favor of learner-centered approaches that actively manage the user's cognitive load during instruction. [9]

Under this framework, learning is modeled not as passive information acquisition or simple response strengthening, but as an active process of knowledge construction.[9] The student acts as an active sense-maker who selects relevant incoming verbal and pictorial information, mentally organizes these elements into coherent cognitive structures, and integrates them with prior knowledge retrieved from long-term memory.[9] In StoryTeller the real-time interleaving of three-dimensional visual assets, written subtitle overlays, and deep generative vocal narration functions as a multimodal scaffolding guide.[9] This design directly supports the qualitative rationale of multimedia instruction by prompting the child's mind to build meaningful, active connections across different representation modalities, thereby fostering deep conceptual transfer over rote memorization.[9]

## Chapter 4. Analysis and Theoretical Foundation

The purpose of this chapter is to outline the theoretical concepts, structural models, and operating principles that form the foundation of the StoryTeller application. Rather than detailing the specific software implementation, this section focuses on the abstract architecture, the chosen communication protocols, the logic behind the generative artificial intelligence integration, and the functional use cases. By defining these structural components, the system's ability to provide a scalable, adaptive, and interactive 3D educational environment is theoretically demonstrated.

### 4.1. Conceptual Architecture and System Overview

To achieve a seamless integration between a real-time interactive 3D environment and advanced natural language processing capabilities, the system architecture is designed using a decoupled Client-Server model. This architectural pattern isolates the heavy graphical rendering tasks from the complex data processing and artificial intelligence generation logic. The architecture relies on three primary logical nodes:

- **The Client Node (Frontend):** Represents the interactive 3D application. Its primary theoretical responsibility is to manage the state of the simulation, handle user inputs, and render the graphical and auditory feedback.
- **The Middleware Node (Backend Server):** Acts as the central processing unit and bridge. It handles business logic, orchestrates the pedagogical flow, and formats the user's progress into structured programmatic prompts.
- **The External AI Node:** Represents the external Large Language Model (LLM) and Text-to-Speech (TTS) application programming interfaces (APIs) responsible for the raw generation of narrative data and audio streams.

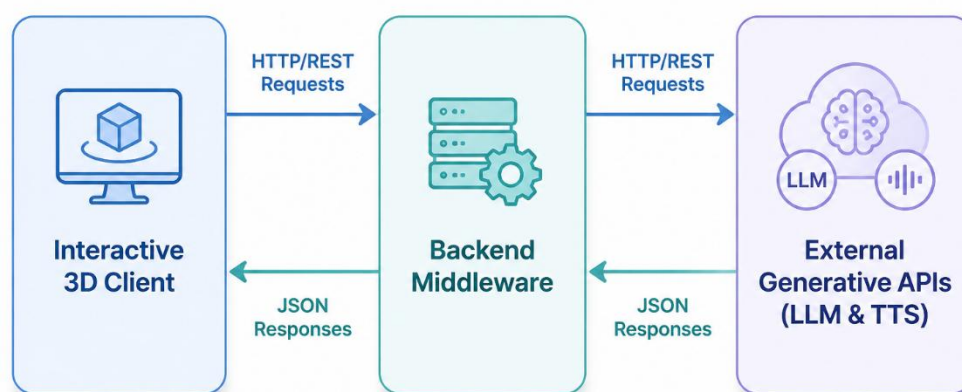


Figure 4.1: High-level conceptual architecture of the StoryTeller system

The communication protocol chosen for data exchange between the Client Node and the Middleware Node is Representational State Transfer (REST) over the standard Hypertext Transfer Protocol (HTTP). A RESTful theoretical approach ensures that the communication remains stateless; every request sent from the client contains all the necessary information for the server to understand and process it, without requiring the server to store session contexts.

Data serialization across this pipeline is uniformly handled using the JavaScript Object Notation (JSON) format. JSON was selected as the abstract data model due to its lightweight nature, human-readability, and strict hierarchical structure, which is essential for parsing dynamically generated quizzes and narrative branching logic without losing structural integrity.

## 4.2. Functional Structure and Use Case Modeling

Before defining the technical data structures, it is necessary to abstract the system's functional requirements using Unified Modeling Language (UML) Use Case modeling. For this educational platform, the functional logic defines three distinct actors interacting within a Learning Management System (LMS) paradigm:

- **The Educator (Administrative Actor):** The user responsible for orchestrating the pedagogical process. Their theoretical goals include defining lesson parameters, uploading curriculum constraints, and accessing aggregated statistical dashboards to monitor student performance and grading metrics.
- **The Student (Primary Actor):** The end-user interacting with the 3D graphical interface. Their goals include navigating the virtual environment, experiencing the narrative, and solving dynamically generated educational assessments.
- **The AI Provider (Secondary/External Actor):** The external automated system that processes contextual prompts and returns generated educational content based on the educator's constraints.

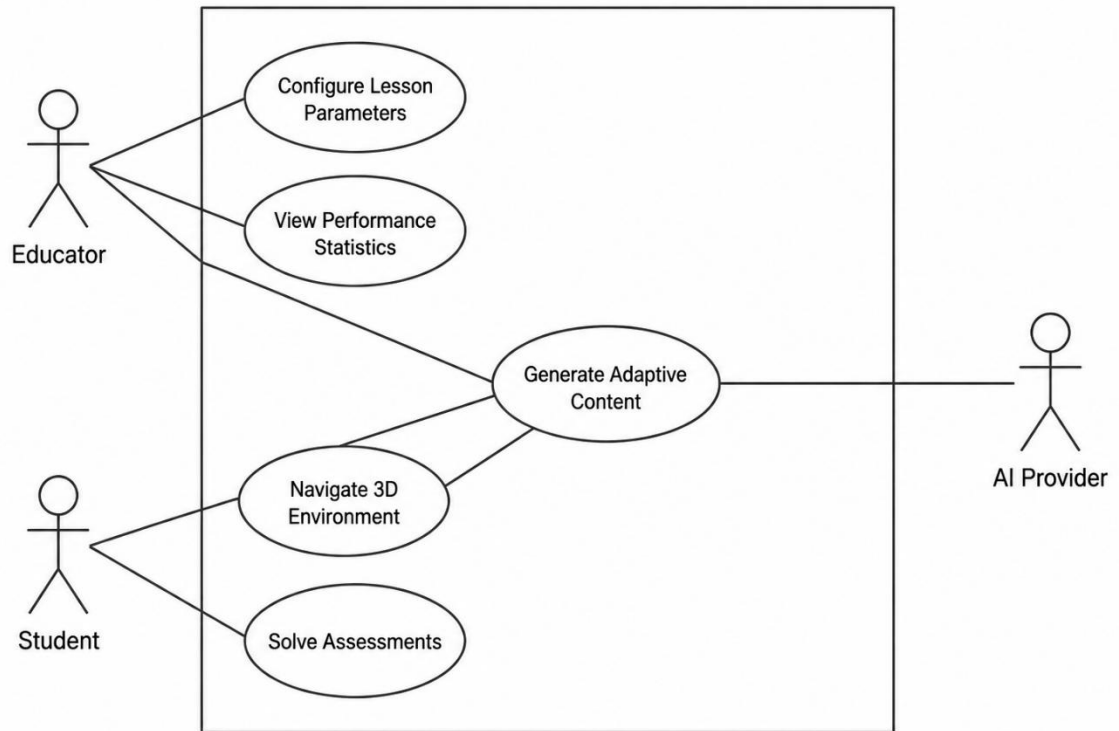


Figure 4.2: UML Use Case diagram illustrating system interactions

The functional logic dictates that the system operates in a reactive loop. The primary actor triggers an event (entering a specific zone in the virtual environment). This action forces the system to compile a contextual prompt, which is transmitted to the secondary actor. Once the external actor returns the generated scenario, the system parses the data, updates its internal logical state, and presents the new challenge to the student, thereby closing the interactive loop.

### 4.3. Learning Management Logic and Abstract Data Models

To support the educator's administrative capabilities, the system relies on structured data models that track and aggregate user interactions. The backend middleware operates theoretically as an orchestration engine, bridging the gap between the educator's input and the student's 3D experience.

The theoretical data flow for the learning management process involves three core components:

- **Content Orchestration:** Translating the educator's high-level lesson topics into strict configuration constraints. These constraints dictate the boundaries within which the AI is allowed to generate narrative events.
- **Session Tracking and Telemetry:** As the student interacts with the 3D environment, the system abstracts these interactions into discrete data points, such as time spent per narrative node and attempt rates for generated quizzes.
- **Performance Aggregation:** The middleware evaluates the student's responses, calculates normalized grades, and structures this raw data into aggregated statistical overviews. These metrics are subsequently exposed to the educator through a dedicated dashboard interface.

This continuous feedback loop ensures that the generative AI serves a strict pedagogical purpose, operating entirely within the instructional parameters defined by the educator while providing real-time analytics.

#### 4.4. Prompt Engineering Logic and AI Constraint Mechanisms

To generate contextually accurate and pedagogically safe content, the middleware relies on advanced prompt engineering techniques. The interaction with the external LLM is not a standard conversational exchange; rather, it is a highly structured programmatic request designed to constrain the AI's output and enforce deterministic behavior. The theoretical model for this communication employs a bipartite prompt structure:

- **The System Directive:** This immutable layer establishes the AI's operational persona ( a strict educational evaluator). It defines the overarching rules, mandates the tone of the narrative, and explicitly prohibits the generation of content outside the established curriculum boundaries (hallucination mitigation).
- **The Contextual Payload:** This dynamic layer contains the specific variables injected by the system at runtime. It includes the Educator's constraints (subject matter, target difficulty) and the Student's current state within the 3D environment.

A critical theoretical challenge in utilizing generative AI for software integration is ensuring that the output is machine-readable. To achieve this, the prompt logic incorporates abstract schema definitions. By explicitly defining the expected output keys in the prompt, the LLM is forced to map its natural language generation into predictable key-value pairs (mapping the narrative to a `story_text` key, and the assessment to a `quiz_options` array).

| Parameter Category         | Description                                                                           | Pedagogical Purpose                                                      |
|----------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Curriculum Context</b>  | The overarching subject matter and specific topic parameters defined by the Educator. | Ensures the generated narrative aligns with the educational goals.       |
| <b>Difficulty Modifier</b> | A dynamic variable ( beginner, advanced) adjusting vocabulary and quiz complexity.    | Provides adaptive scaffolding based on the Student's prior performance.  |
| <b>Format Constraint</b>   | Explicit instructions dictating the output structure (strictly JSON format).          | Guarantees the backend middleware can parse the data without exceptions. |
| <b>Validation Rules</b>    | Logical constraints ("provide exactly one correct answer out of four options").       | Maintains the structural integrity of the generated assessment.          |

Table 4.1: Prompt Payload Parameters and Pedagogical Constraints

By structuring the requests using these abstract parameters, the middleware effectively bridges the gap between unpredictable generative models and the strict logic required by the interactive 3D application.

#### 4.5. Finite State Machine (FSM) Logic of the Interactive Client

To manage the complex flow of events within the 3D environment without encountering race conditions or logical deadlocks, the client application operates theoretically on a Finite State Machine (FSM) pattern. This algorithmic model ensures that the educational application can only exist in exactly one predefined state at any given moment, strictly controlling what inputs are accepted and what actions are executed.

The theoretical FSM designed for the StoryTeller client comprises five primary states:

1. **Exploration State:** The default state where the Student is granted full mechanical control to navigate the 3D environment. Background processes (such as rendering and physics calculations) run normally, but no educational progression is actively processed.
2. **Trigger State (Generation Wait):** Activated when the Student collides with a designated narrative hotspot. The system suspends user movement, displays a loading interface, and initiates the asynchronous REST API call to the backend middleware.
3. **Narrative State:** Once the JSON response is received and parsed, the system transitions into this state. The generated text is rendered to the user interface, and the synchronized TTS audio stream begins playback. User input is restricted to advancing the dialogue.
4. **Assessment State:** Automatically triggered after the narrative delivery concludes. The UI presents the dynamically generated multiple-choice quiz. The system activates an internal timer to measure response latency and records the student's selected answer.
5. **Telemetry Sync State:** Following the assessment, the system packages the interaction data (time spent, choice accuracy) and transmits it back to the middleware for the Educator's dashboard. Upon successful transmission, the FSM returns to the Exploration State.

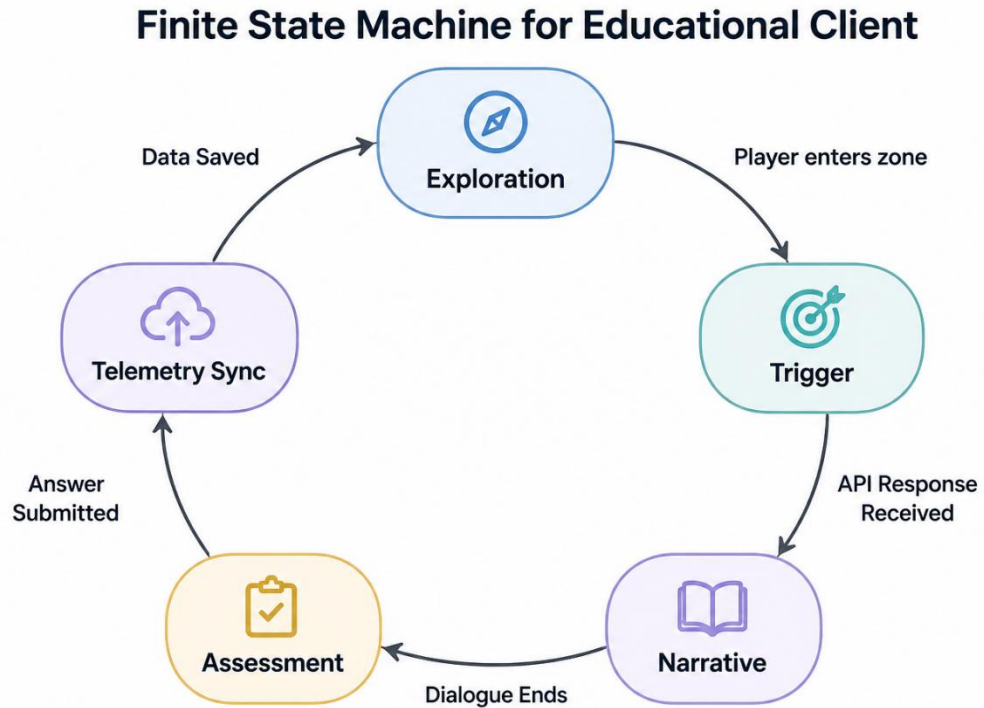


Figure 4.3: Educational Client State Transition Diagram

#### 4.6. Abstract Entity Relationship and Data Propagation

While the backend uses a non-relational or document-based approach to handle dynamic AI content, the theoretical relationships between the core data entities remain strictly structured. The system relies on three interconnected abstract models to maintain consistency between what the Educator defines and what the Student experiences.

1. **The Educator Configuration Entity:** This model acts as the root constraint. It holds the overarching parameters for a specific learning module, including the subject identifier, the targeted difficulty level, and the maximum number of narrative events allowed per session.
2. **The Generation Payload Entity:** A transient data structure created dynamically during the interaction loop. It inherits the constraints from the Educator Configuration and encapsulates the newly generated AI content, specifically the story text string and an array of possible quiz answers, out of which strictly one is flagged as correct.
3. **The Session Telemetry Entity:** A historical data model constructed after the Student interacts with the Payload Entity. It records the interaction timestamp, the specific prompt that was generated, the user's selected answer, and a boolean value indicating success or failure.

| Entity Name                   | Creation Trigger                                           | Primary Abstract Attributes                                         |
|-------------------------------|------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Educator Configuration</b> | Manual input by the Educator prior to session start.       | Subject ID, Topic Keywords, Difficulty Constraint, Event Limit.     |
| <b>Generation Payload</b>     | Asynchronous API response triggered by Student navigation. | Narrative String, Quiz Prompt, Options Array, Correct Answer Index. |
| <b>Session Telemetry</b>      | Student submits an answer during the Assessment State.     | Timestamp, Time Elapsed, Selected Index, Accuracy Flag.             |

Table 4.2: Data Entities and Lifecycle Breakdown

### 4.7. Adaptive Scoring Algorithm Logic

A significant theoretical challenge in a system where assessments are dynamically generated by an LLM is establishing a fair and consistent grading mechanism. Because the exact questions are not hard-coded, standard static grading algorithms are insufficient. To solve this, StoryTeller theoretically implements an Adaptive Scoring Algorithm.

The algorithmic logic relies on a weighted calculation rather than raw point accumulation. When the system evaluates a submitted answer, it processes three specific variables:

- **Accuracy (Binary):** Determines if the selected answer matches the correct index provided by the AI payload.
- **Difficulty Weight (Float):** A multiplier extracted from the Educator Configuration Entity. Advanced topics yield higher base rewards than beginner topics, incentivizing progression.
- **Temporal Penalty (Float):** A reduction factor calculated based on the time elapsed between the presentation of the assessment and the submission of the answer. If the response time exceeds a pre-calculated optimal threshold, the rewarded score is incrementally reduced to prevent guessing or unwarranted delays.

By abstracting the grading process into these three interdependent variables, the system ensures that the Educator receives a normalized performance metric. This normalized score accurately reflects the student's cognitive engagement and comprehension speed, regardless of the unpredictable nature of the generative AI's specific phrasing.

### 4.8. Threat Modeling and AI Prompt Injection Mitigation

Integrating Large Language Models into interactive client-facing applications introduces unique theoretical vulnerabilities that do not exist in traditional deterministic software. The primary vulnerability in the StoryTeller architecture is "Prompt Injection," a scenario where the primary actor (the Student) intentionally provides adversarial input designed to override the system's underlying directives or bypass the Educator's curriculum constraints.

To maintain the pedagogical integrity of the platform, the middleware implements a theoretical three-stage mitigation pipeline:

1. **Input Sanitization Phase:** Before any user-generated data or contextual parameters are merged into the Prompt Payload, the middleware scrubs the data of potential escape characters and explicit override commands (phrases like "ignore previous instructions").
2. **Contextual Boundary Enforcement:** The System Directive (as defined in Section 4.4) is strictly isolated from the dynamic user context. The model is theoretically instructed to treat all incoming dynamic variables purely as data strings, preventing the LLM from executing them as administrative commands.
3. **Output Validation Parsing:** Because generative models operate probabilistically, the system cannot blindly trust the AI's response. The middleware applies a strict schema validation check. If the returned JSON object contains malformed keys, missing correct answer flags, or content that violates the predefined difficulty length, the response is discarded, and a fallback regeneration request is triggered.

| Threat Vector             | Description                                                   | Logical Countermeasure                                              |
|---------------------------|---------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Prompt Injection</b>   | User attempts to overwrite system rules via input.            | Strict variable isolation and input sanitization before generation. |
| <b>Data Hallucination</b> | AI generates factually incorrect or out-of-bounds curriculum. | Educator-defined context constraints and temperature throttling.    |
| <b>Schema Breakage</b>    | AI returns plain text instead of structured JSON format.      | Output validation parsing and automated fallback regeneration.      |

Table 4.3: Generative Content Threat Modeling

## 4.9. Spatial Collision and Event-Driven Logic

Within the client application, the transition from the "Exploration State" to the "Trigger State" relies on an event-driven spatial architecture. To bridge the gap between the 3D coordinate space and the logical progression of the educational narrative, the system employs abstract bounding volumes (collision detection logic).

Theoretical collision detection operates by defining invisible mathematical boundaries ( Axis-Aligned Bounding Boxes or Sphere Colliders) around specific coordinates in the virtual environment, designated as narrative checkpoints. The client continually monitors the positional matrix of the Student actor. When the coordinates of the primary actor intersect the boundaries of a predefined checkpoint, a spatial event is fired.

This architectural choice decouples the visual rendering of the 3D world from the underlying educational logic. The spatial event acts as a standard interrupt signal, immediately pausing the physics simulation, locking user input, and invoking the REST API request. By handling spatial triggers asynchronously, the application ensures that the 3D simulation remains fluid and responsive, dropping no frames even when preparing to transition states.

#### 4.10. Sequence Modeling of the Generative Interaction

To fully abstract the interaction between the distributed components of the system, a chronological sequence logic must be defined. While the FSM (Section 4.5) describes the internal states of the client, the sequence model maps the asynchronous communication across the network boundary.

The theoretical generation cycle follows a strict synchronous/asynchronous hybrid sequence:

1. **Initiation:** The Client detects a spatial collision event and sends an HTTP POST request containing the localized context to the Middleware.
2. **Orchestration (Asynchronous Wait):** The Middleware acknowledges the request, retrieves the active Educator Configuration, formats the LLM prompt, and dispatches it to the External AI Node. During this time, the Client operates in a loading state.
3. **Processing:** The LLM generates the text, and the TTS engine synthesizes the corresponding audio stream.
4. **Delivery and Validation:** The Middleware receives the raw data, executes the Output Validation Parsing (from Section 4.8), and forwards the validated JSON response back to the Client.
5. **Execution:** The Client parses the JSON, updates the UI, plays the audio, and awaits the user's assessment input, closing the sequence.

Figure 4.4: UML Sequence Diagram of the Generative Interaction Cycle

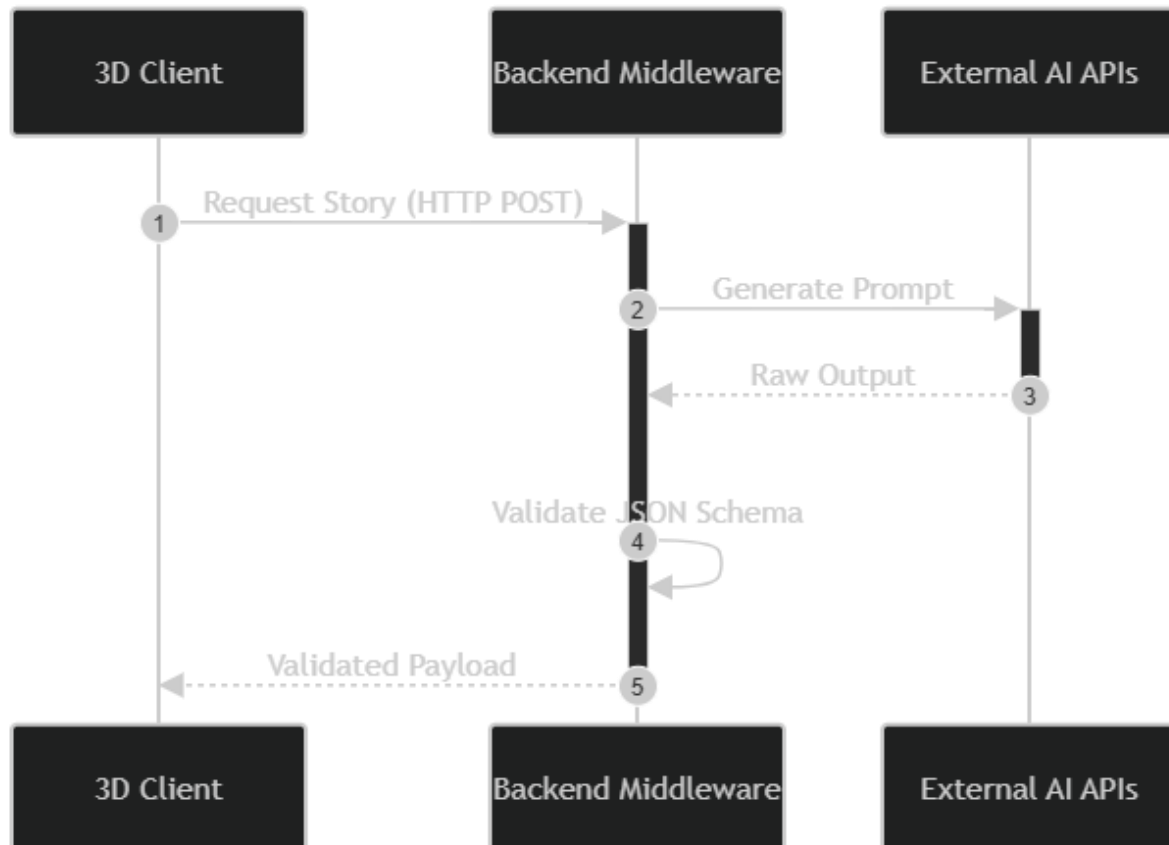


Figure 4.4: UML Sequence Diagram of the Generative Interaction

## 4.11. System Specifications and Constraints

To conclude the theoretical foundation, it is necessary to establish the formal specifications that govern the system's design. These specifications define the expected capabilities and constraints of the StoryTeller platform without dictating the underlying implementation technologies.

### 4.11.1. Functional Specifications

The functional specifications define the absolute behaviors the system must exhibit to fulfill its multi-actor use cases:

- The system must allow an administrative actor to define overarching curriculum constraints, including subject matter and desired difficulty metrics.

- The middleware must aggregate interaction telemetry and compute an adaptive score based on temporal latency and assessment accuracy.
- The client must support spatial navigation in three dimensions and detect collision events to trigger narrative generation.
- The system must synthesize generative text into real-time auditory feedback to accommodate multi-sensory learning paradigms.

#### 4.11.2. Non-Functional Specifications

The non-functional specifications outline the quality attributes and performance constraints required for an optimal user experience:

- **Latency and Responsiveness:** Because the system relies on external LLM and TTS APIs, the middleware must process and return validated payloads within an acceptable threshold (theoretically under a few seconds) to prevent the user from disengaging from the 3D simulation.
- **Scalability:** The RESTful, stateless nature of the middleware ensures that the system can theoretically handle concurrent requests from multiple distinct Student actors without memory cross-contamination.
- **Data Integrity:** All telemetric data must be recorded with strict atomicity. If a student disconnects during an assessment state, the system must logically discard the incomplete session to prevent polluting the educator's statistical dashboard with malformed scores.
- **Modularity:** The separation of the 3D presentation layer from the generative logic ensures that either component can be upgraded or substituted without requiring a total architectural rebuild.

## Chapter 5. Detailed Design and Implementation

The purpose of this chapter is to document the developed StoryTeller application comprehensively, ensuring that the system can be maintained, scaled, and further developed by future engineers. This section transitions from the theoretical models established in Chapter 4 to the concrete technological stack, detailing the implementation of every component at the module level, illustrating class diagrams, extracting key methods from core classes, and defining the database architecture.

### 5.1. General Application Architecture and Technology Stack

The StoryTeller platform is built upon a decoupled, distributed architecture that separates the heavy 3D graphical processing from the asynchronous artificial intelligence generation logic. This design pattern was chosen to prevent the Unity main thread from freezing during the high-latency HTTP requests required by external Large Language Models (LLMs).

The technology stack selected for the implementation comprises:

- **Frontend Engine:** Unity 3D (C#), utilizing Cinemachine for dynamic camera tracking and DOTween for procedural animation smoothing.
- **Backend Orchestrator:** Python 3, executed via the Windows Subsystem for Linux (WSL), handling OS-level file manipulation, string sanitization, and API orchestration.
- **External APIs:** Google Gemini API (gemini-2.5-flash) for narrative and JSON generation, Google Text-to-Speech (gTTS) for audio synthesis, and Pollinations AI for dynamic texture generation.



Figure 5.1: Entity-Relationship Diagram of Database Schema

## 5.2. Database Design and Data Persistence

To meet the requirements of the Learning Management System (LMS) without introducing the overhead of a heavy relational database server (PostgreSQL or MySQL), the application implements a lightweight, document-based NoSQL architecture. Data persistence is achieved through localized JSON files stored directly within Unity's `Application.persistentDataPath`, ensuring immediate read/write access and high portability.

The database is structured into four primary directory nodes, each acting as a distinct collection:

1. **Users Directory:** Stores authentication and role-based access control (RBAC) data.
2. **Classes Directory:** Acts as the relational mapping node between Teachers and Students.
3. **Student Progress Directory:** Stores complex telemetric data arrays, tracking attempts, status (Locked/Unlocked), and score history.

4. **Teacher Repository:** A physical file storage architecture where generated digital assets (JSON payloads, .mp3 audio, .jpg textures) are saved for offline retrieval.

To illustrate the database design, the following JSON structure represents the ClassData schema, demonstrating how students and generated lessons are mapped to a specific educator:

```
{
  "idClasa": "grupa_mica_engleza",
  "idProfesor": "profesor_ionescu",
  "idElevi": ["elev_maria", "elev_andrei"],
  "lectiiGenerate": [ "animale_salbatice", "culori_de_baza" ]
}
```

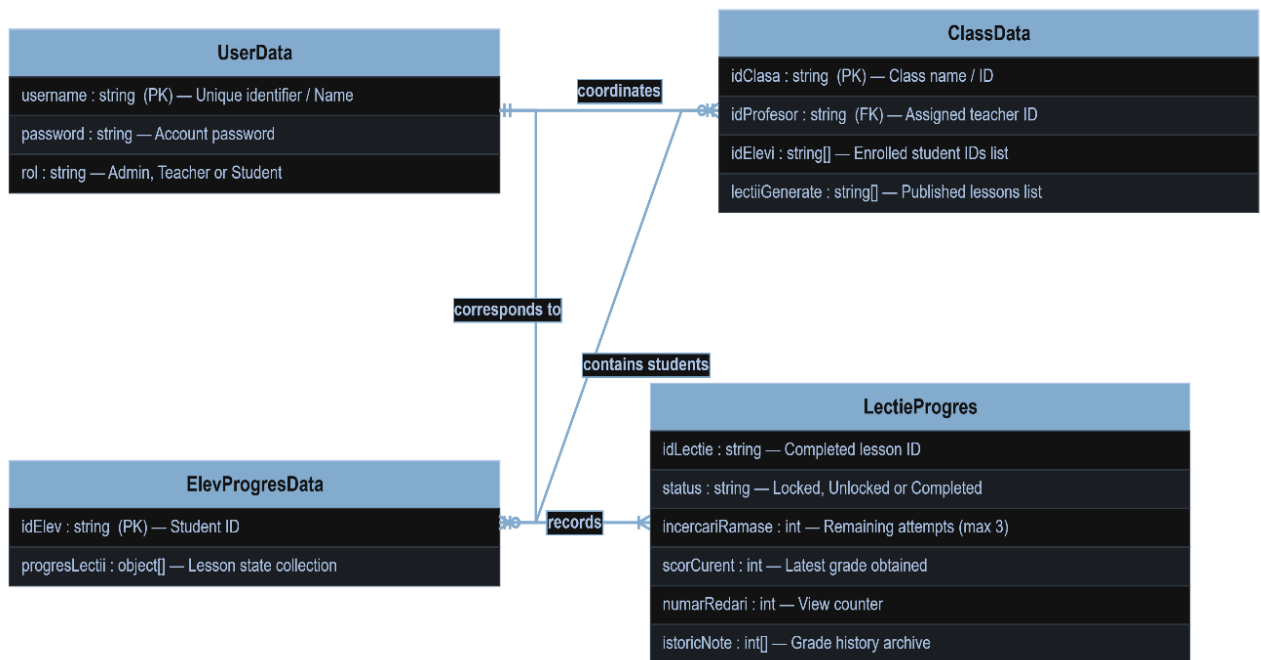


Figure 5.2: Document-Oriented Database Schema Architecture

### 5.3. Backend Generative Module (Python Implementation)

The Python backend module (`generator_poveste.py`) is the core engine responsible for data synthesis. It operates entirely autonomously once triggered by the Unity client, receiving its execution parameters through standard command-line arguments (`sys.argv`).

#### 5.3.1. Prompt Engineering and JSON Schema Enforcement

The `prompt_iningeresc` method is the most critical algorithmic component of the backend. It uses string interpolation to dynamically inject the educator's constraints (theme, vocabulary

size, quiz options) into a strict system directive. To guarantee that the Unity C# deserializer does not crash, the prompt enforces a rigid JSON schema in the response.

```
def prompt_ingineresc(tema, hero_name, dimensiune_quiz, variante_quiz, l_baza, l_inv):
    durata_poveste_minute = 3
    numar_imagini = durata_poveste_minute * 2

    return f"""
    You are an adaptive Educational Director.
    TASK: Create a unique story based EXCLUSIVELY on "{tema}".

    JSON STRUCTURE:
    {{
    "actors": [ {{ "id": "hero", "prefabName": "{hero_name}" }} ],
    "actions": [
    {{ "type": "SPEAK", "text": "...", "animation": "doWave" }},
    {{ "type": "SHOW_IMAGE", "imagePrompt": "..." }}
    ],
    "quiz": [
    {{
    "question": "Question text?",
    "options": ["Opt 1", "Opt 2"],
    "correct_index": 0,
    "tipIntrebare": "Text"
    }}
    ]
    }}
    Return ONLY JSON.
    """
```

Code Snippet 5.1: Enforcement of the JSON schema in the LLM prompt

### 5.3.2. Sequential Image Generation Guarantee

One of the major implementation challenges was preventing asynchronous race conditions when downloading multiple generative images. If the LLM returned arbitrary image names, the files could overwrite each other. The `genereaza_imagini_securizat` method solves this by ignoring the AI's file naming attempts and imposing a strict sequential counter (`image_1.jpg`, `image_2.jpg`), ensuring absolute referential integrity between the JSON data and the physical files.

```
def genereaza_imagini_securizat(story_data):
    contor_secvential = 1
    actions = story_data.get("actions", [])

    for action in actions:
        if action.get("type") == "SHOW_IMAGE" and action.get("imagePrompt"):
            prompt = action["imagePrompt"]
            nume_secvential = f"image_{contor_secvential}.jpg"

            # Synchronous HTTP GET request to Pollinations AI API
```

```
if descarga_si_salveaza_pollinations(prompt, nume_secvential):
    action["imageFile"] = nume_secvential
    salveaza_progres_json(story_data)

    contor_secvential += 1
return story_data
```

Code Snippet 5.2: Sequential asset mapping logic

## 5.4. Frontend Orchestration and IPC Implementation (*GameInterface.cs*)

The *GameInterface* class serves as the primary configuration and setup interface layer of the Unity frontend. Its main architectural responsibility is capturing user inputs such as the desired narrative theme, hero selection, session duration, age group, base language, and learning language and orchestrating the asynchronous launch of the Python backend middleware.

To prevent interface freezes during high-latency operations, the module implements Inter-Process Communication (IPC) via an asynchronous coroutine named *GenerareFlux*. Rather than halting the application, it instantiates a system process using the *ProcessStartInfo* class, targeting the Windows Subsystem for Linux (*wsl.exe*) to execute the Python script.

The coroutine implements an asynchronous polling loop (`while (!process.HasExited) { yield return null; }`). This yields execution back to the Unity engine frame scheduler, ensuring that graphical rendering and interface animations continue uninterrupted while the backend processes the generative pipeline.

## 5.5. Central Orchestrator and LMS Controller (*StoryDirector.cs*)

The *StoryDirector* class serves as the central control unit and Learning Management System (LMS) orchestrator of the platform. It bridges administrative management, secure user authentication, asset persistence, and narrative execution. Rather than handling raw UI inputs directly, it acts as a state coordinator that routes users based on Role-Based Access Control (RBAC) into specific administrative, pedagogical, or student dashboards.

The operational workflow of the LMS controller encompasses database bootstrapping, role verification, and the sequential execution of narrative actions loaded from the NoSQL document storage.

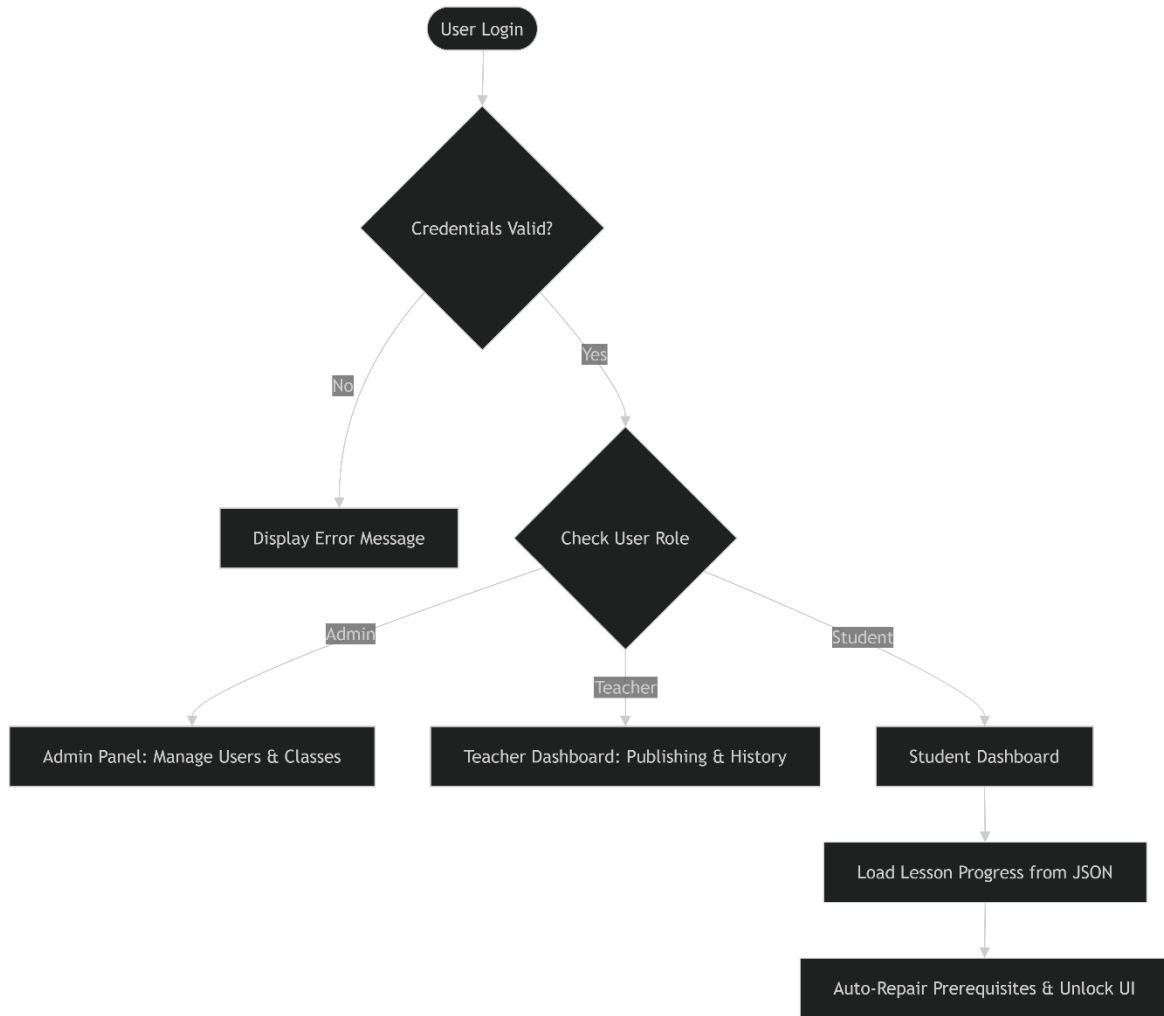


Figure 5.3: Role-Based Access Control Workflow Architecture

### 5.5.1. Sequence Execution and Asset Coordination

Once a lesson is initiated, the director deserializes the `story.json` payload and coordinates synchronous and asynchronous operations. It coordinates multiple subsystems simultaneously: adjusting Cinemachine camera priorities (`vcamGeneral` vs. `vcamMonitor`), triggering skeletal animations via Unity's Animator component, loading textures dynamically onto 3D display monitors, and yielding execution until speech synthesis audio clips (`gTTS`) finish playing.

### 5.5.2. Component Configuration (StoryDirector)

The central orchestration of the 3D scene, UI bindings, and Role-Based Access Control (RBAC) panels is configured directly within the Unity editor through the StoryDirector component attached to the main Director GameObject. This component exposes a structured set of serialized fields that link runtime logic directly to scene elements:

- **Setup Scene / Library Prefabs:** Manages the dynamic instantiation of character models, including Element 0: Boy and Element 1: Anime\_Girl, allowing the system to populate the 3D space based on generative metadata.

- **Camera and Audio Subsystems:** Binds Vcam General and Vcam Monitor (utilizing Cinemachine camera blending for smooth cinematic transitions) alongside the primary Sursa Audio source and the Quiz Manager interface.
- **Interface and Screen Systems:** Connects runtime text components (Text Subtitrare via TextMeshPro) and render textures (Magic Screen) to display dynamically generated visual assets.
- **LMS Panels and Access Control (RBAC):** Links administrative and educational interface elements directly to the script, grouping references for the Admin Panel, Profesor Panel, and Elev Panel, alongside user management inputs, class allocation dropdowns, and history containers for publishing custom lessons.

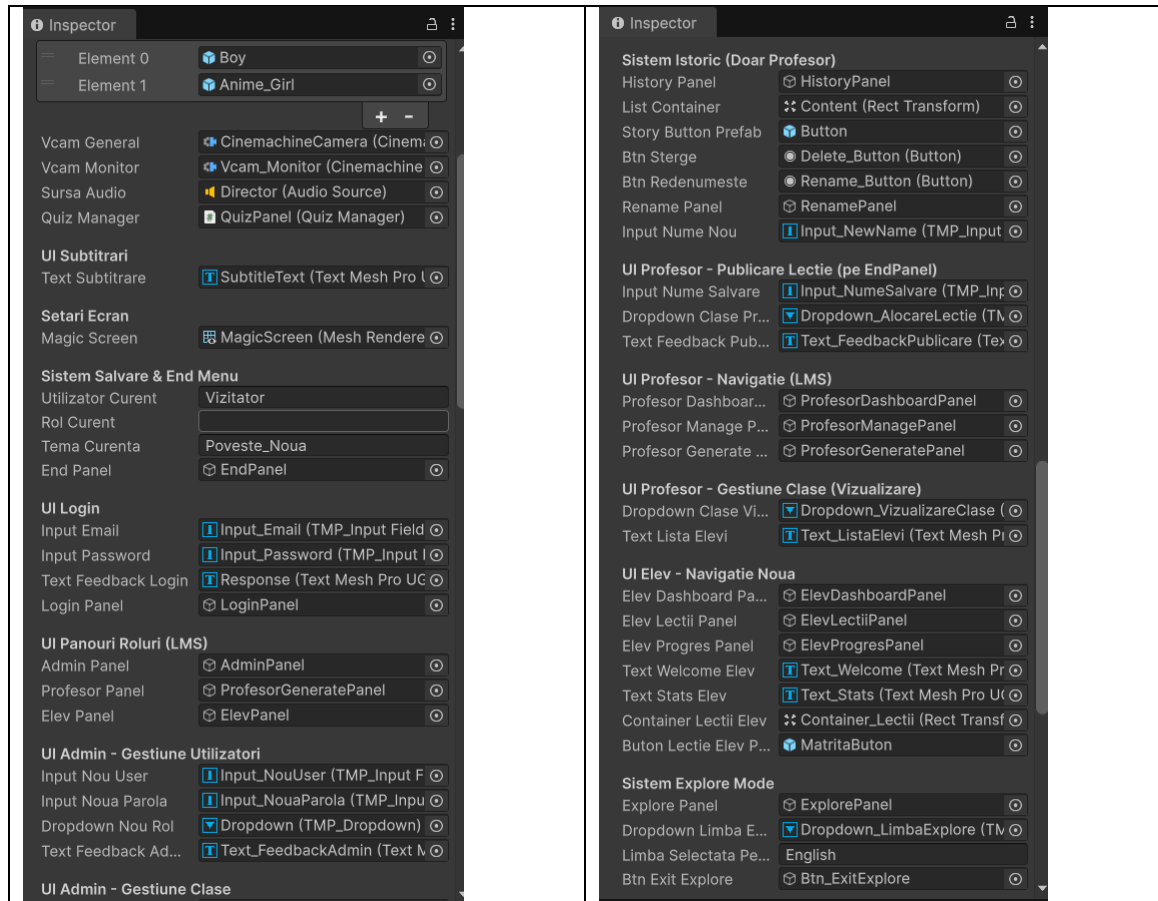


Figure 5.4: StoryDirector Inspector Configuration

### 5.5.3. Authentication and Role-Based Login Interface

Upon launching the application, users are greeted by the authentication panel illustrated in Figure 5.7. This interface manages secure entry into the platform by enforcing Role-Based Access Control (RBAC) against the local NoSQL user records stored in Unity's persistent data path. The authentication screen provides the following key UI components:

- **Input Fields:** Dedicated text boxes for entering the user credentials, specifically the username ("Enter user...") and password ("Enter pass...").
- **Confirm Button:** Validates the entered credentials against the stored JSON schema, determining the user's role and routing them dynamically to the appropriate dashboard (Admin, Teacher, or Student).

- **Create New Button:** Allows the registration or provisioning of new user accounts within the local database structure.
- **Clear Button:** Resets the input fields to clear current text entries.
- **Feedback Display:** A dedicated text element ("Feedback") providing real-time validation alerts, error messages, or success confirmations during the login sequence.

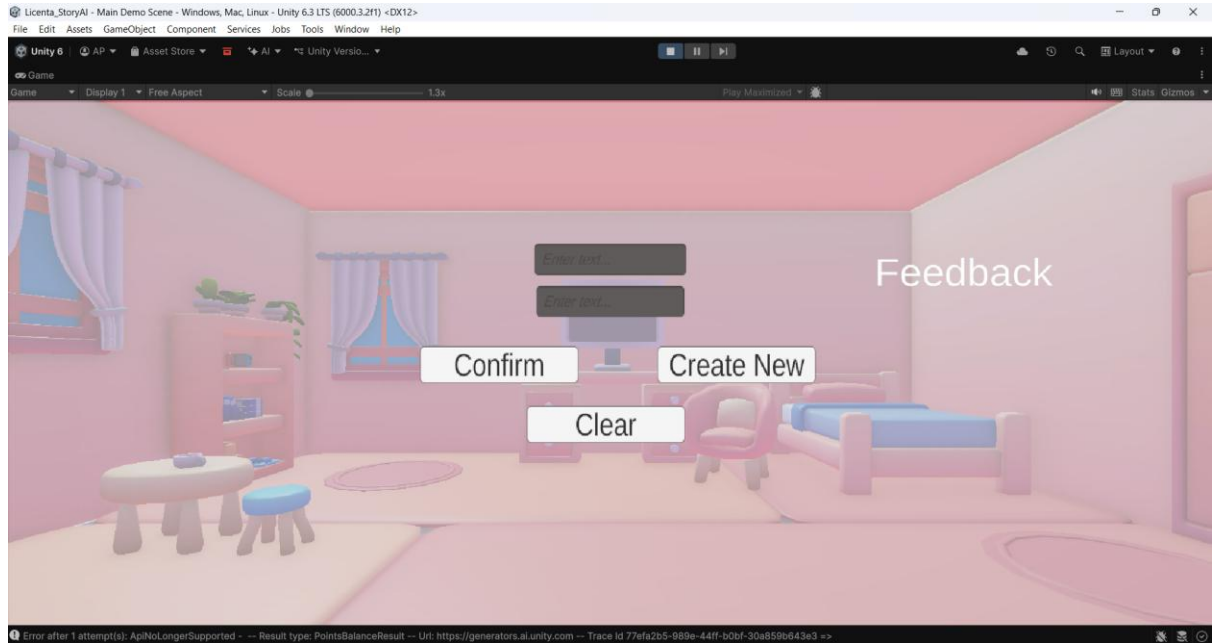


Figure 5.5: Authentication and User Login Interface

#### 5.5.4. Educator Dashboard and Class Student Monitoring Interface

Once authenticated as an educator, the user gains access to the management and monitoring interface illustrated in Figure 5.9. This module bridges the administrative database nodes with the instructor's workflow, providing real-time visibility into classroom organization and student distribution. The management screen incorporates the following core functional elements:

- **Class Selector Dropdown:** A selection menu allowing the educator to switch between managed classes (selecting "sincai\_3b") to query the underlying relational mapping records.
- **Student List Display ("Elevii"):** An interactive text container that dynamically populates and lists all students enrolled in the selected class (such as "IONUT\_POP", "ONA\_MIHAI", and "GLIGAN\_DAVID"), reflecting the data mapped within the local NoSQL ClassData documents.
- **Navigation Control ("Back"):** A button enabling the educator to return to the primary dashboard panel seamlessly.



Figure 5.6: Educator Dashboard for Student Records

### 5.5.5. Teacher Generation Interface and Parameter Configuration

The educator's workflow is centered around a dedicated interactive generation panel, as illustrated in Figure 5.7. This interface allows authenticated teacher accounts to define custom pedagogical parameters without requiring direct code manipulation or technical intervention. The panel provides input fields and dropdown controls for specifying:

- **Narrative Theme and Prompt Input:** A text field allowing educators to input specific curriculum themes or story prompts ("*Scrive povestea ta aici...*").
- **Duration Slider:** A temporal control slider adjusting the estimated session length (configured to 2 minutes).
- **Character Selection Dropdown:** A selector enabling the educator to choose the primary avatar model (Boy or Anime\_Girl).
- **Language and Localization Configuration:** Dropdown options for defining both the base language and the target learning language.
- **Assessment Parameters:** Controls for configuring quiz dimensions and variant counts.
- **Execution Trigger:** The primary green **GENEREAZA** button, which initiates the asynchronous Inter-Process Communication (IPC) pipeline via WSL to execute the Python generative backend.

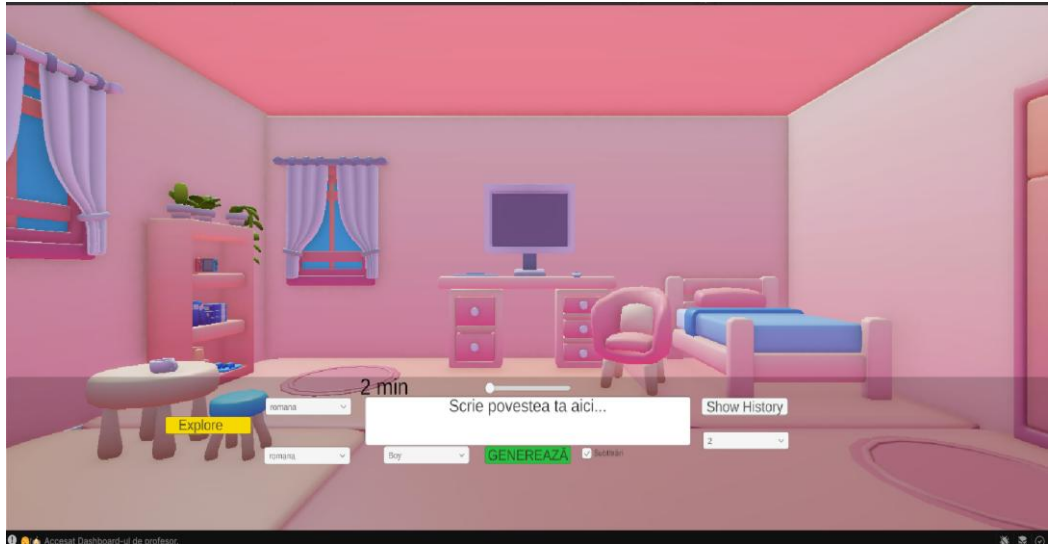


Figure 5.7: Custom Lesson Generation Dashboard

## 5.6. Deterministic Physics and Camera Control (*ActorController.cs*)

The ActorController module manages the physical movement and spatial interaction of the 3D avatar within the interactive "Explore" mode. To ensure architectural stability and prevent null-reference exceptions at runtime, the module enforces a strict dependency rule using the `[RequireComponent(typeof(Rigidbody))]` attribute, guaranteeing that a physical mass body is automatically coupled to the avatar.

To decouple simulation mechanics from variable hardware frame rates, movement and rotational calculations are isolated entirely within the `FixedUpdate()` method rather than the standard `Update()` loop. This ensures that spatial displacement is synchronized strictly with the deterministic timing of the NVIDIA PhysX engine.

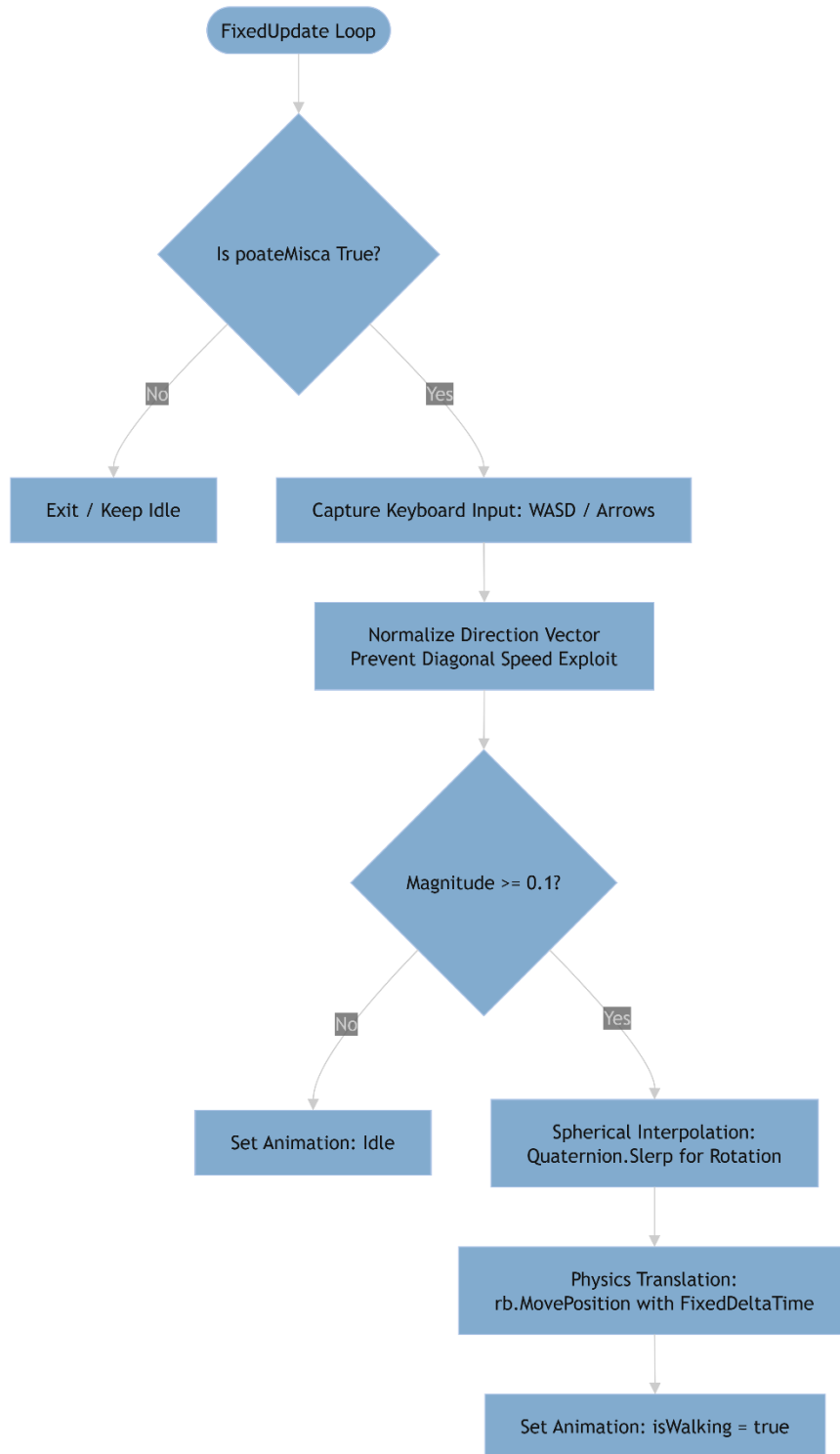


Figure 5.8: Physics Update Loop and Vector Normalization

## 5.7. Dynamic Assessment Module (QuizManager.cs)

The QuizManager class is responsible for the runtime instantiation and interactive evaluation of dynamic assessments. Because questions and answers are generated programmatically by the backend AI without hardcoded templates, the module handles polymorphic question types including standard text queries, statement images (TextImagine), and multi-image visual options (ImagineAB).

Asset conditioning and texture mapping are handled asynchronously at runtime via UnityWebRequestTexture, mapping downloaded binary streams directly onto UI raw image components. When an end-user submits a response, the verification logic checks the selected index against the AI-generated correct\_index, assigns localized feedback strings based on the user's base language preferences, and reports final telemetric grades back to the StoryDirector to update the student's persistent progress collection.

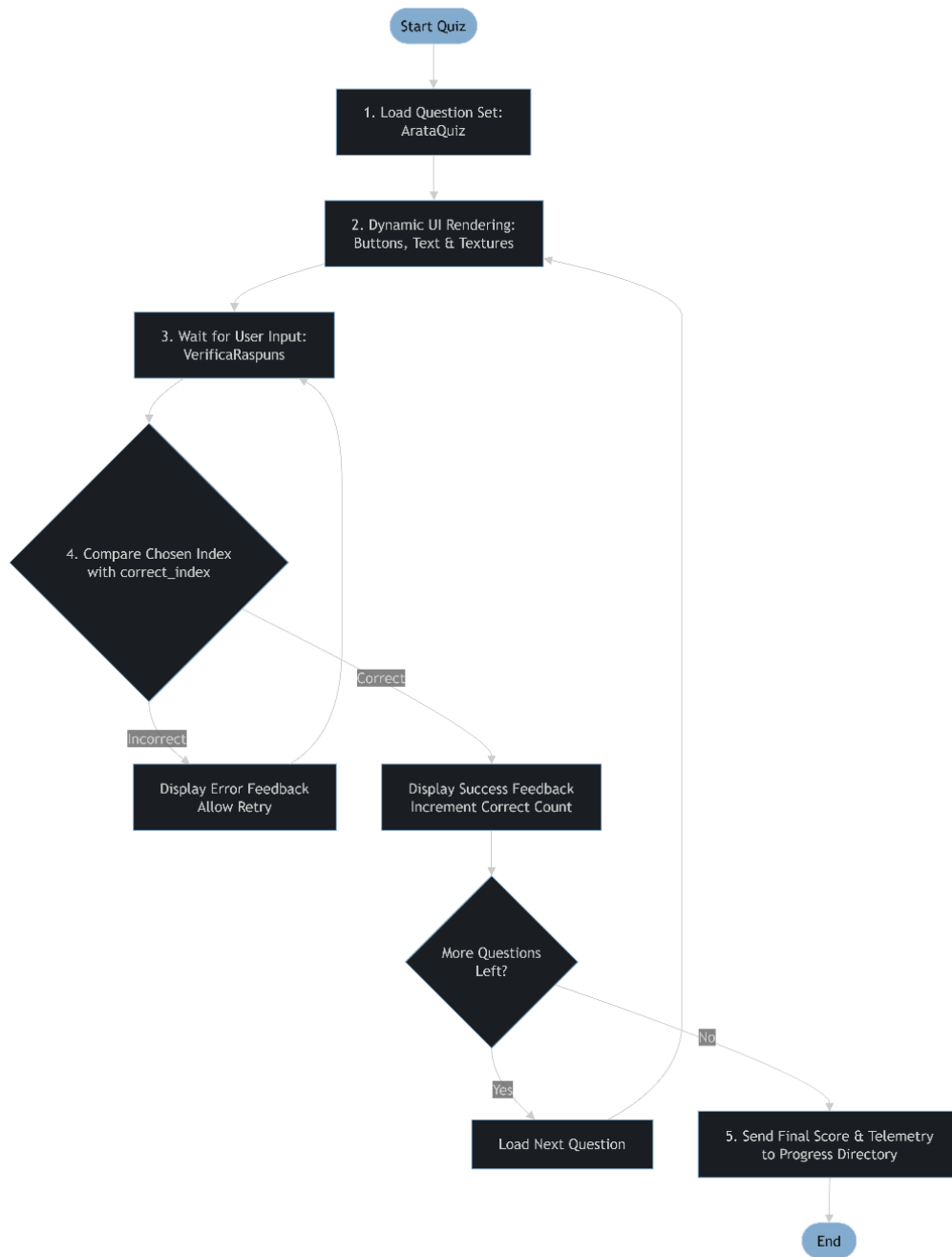


Figure 5.9: Dynamic Assessment and Telemetry Feedback Flow

## 5.8. Audio Synthesis, Streaming, and Subtitle Interleaving

A critical requirement of an immersive educational storytelling platform is the tight temporal synchronization between spoken narration, textual subtitles, and 3D visual cues. The platform achieves this through a hybrid pipeline combining server-side text-to-speech (TTS) generation in Python and asynchronous audio streaming in Unity.

### 5.8.1. Server-Side Audio Generation (genereaza\_audio)

In the Python backend module (generator\_poveste.py), once the textual narrative actions are synthesized by the Large Language Model, the genereaza\_audio function iterates through each action block and invokes the Google Text-to-Speech (gTTS) library.

- Text strings are converted into binary MP3 streams using language-specific mapping (matching base language configurations such as Romanian "ro" or English "en").
- Files are sequentially named (audio\_0.mp3, audio\_1.mp3, etc.) and saved directly into the Unity StreamingAssets directory.
- To prevent data corruption during long-running generation tasks, salveaza\_progres\_json() is triggered after each successful audio file write.

### 5.8.2. PlayDynamicAudio Implementation

Within the Unity C# environment, the StoryDirector class handles the playback synchronization via the PlayDynamicAudio coroutine.

- **Dynamic Subtitle Rendering:** Upon executing a narrative action, the spoken text string is immediately assigned to the textSubtitrare UI component, ensuring visual accessibility.
- **Asynchronous Web Requests:** Audio clips are loaded from disk using UnityWebRequestMultimedia.GetAudioClip, utilizing the file:// protocol coupled with path normalization to ensure cross-platform compatibility.
- **Temporal Blocking:** The coroutine uses yield return new WaitForSeconds(clip.length) after invoking sursaAudio.PlayOneShot(clip), ensuring that the narrative sequence pauses deterministically until the spoken audio clip finishes playing before transitioning to the next action block.

## 5.9. Fault Tolerance and Graceful Degradation (AI\_FAILED Architecture)

Distributed systems relying on external cloud APIs (such as Google Gemini and Pollinations AI) are inherently vulnerable to network latency, rate limits (HTTP 429), and service interruptions. To prevent the Unity client from hanging indefinitely or throwing unhandled null-reference exceptions when an AI request fails, a robust fail-safe mechanism was implemented.

### 5.9.1. Backend Exception Handling and Error Flagging

In generator\_poveste.py, the core execution loop (ruleaza\_ai) attempts to query multiple fallback models (gemini-3.1-pro-preview, gemini-2.5-flash, gemini-2.0-flash) across multiple retry iterations with exponential backoff. If all model tiers and retry attempts fail due to timeouts or API rejection:

- The execution pipeline aborts further asset generation.
- Instead of leaving a malformed or empty file, the script writes a strict JSON payload directly to story.json: {"error": "AI\_FAILED"}.

### 5.9.2. Frontend Error Detection (LoadAndRunStory)

The Unity frontend intercepts this condition inside the LoadAndRunStory coroutine within StoryDirector.cs. Upon deserializing the JSON payload, the system checks for the presence of the error token:

```
// Excerpt from StoryDirector.cs: Anti-lock system and error interception
string jsonContent = File.ReadAllText(filePath);
```

```
if (jsonContent.Contains("AI_FAILED"))
{
    Debug.LogError("Generation failed. Fallback file detected.");
    ArataEndPanel();
    yield break; // Graceful exit preventing runtime crashes
}
```

# Code Snippet 5.3 Frontend Error Detection

This degradation ensures that the application catches network anomalies cleanly, exits the narrative sequence safely, and returns the user to the dashboard or end panel without destabilizing the game engine.

## 5.10. Asset Lifecycle and Cache Management

Because the platform generates custom graphical assets and audio streams dynamically on-the-fly for every new lesson, managing disk space and preventing asset cross-contamination between sessions is paramount.

### 5.10.1. Cache Purging (CurataImaginiVechi)

Prior to initiating a new generation cycle or launching an exploration mode, the CurataImaginiVechi() method purges all legacy temporary assets from the working directory:

- It scans Application.streamingAssetsPath for files matching wildcard patterns (\*.jpg and \*.mp3).
- Using File.Delete(), it removes outdated textures and audio files, ensuring that storage bloat is eliminated and that subsequent rendering passes only load fresh, currently mapped files.

### 5.10.2. Teacher Repository Archiving

When a teacher elects to save and publish a newly generated lesson, StoryDirector.SaveCurrentStory() transitions the temporary cache into a permanent structured archive:

- It creates a unique subdirectory inside the teacher's physical drawer (Sertar\_Professori/<utilizator>/<titluSigur>).
- It copies the active story.json manifest along with all associated .jpg textures and .mp3 audio files into the permanent repository, ensuring complete offline portability and historical persistence.

## 5.11. Advanced Telemetric Tracking and Prerequisite Auto-Repair

The educational backend embedded within StoryDirector.cs features an automated progression control system that governs how students advance through assigned curricula.

### 5.11.1. Prerequisite Auto-Repair Algorithm (Elev\_IncarcaLectii)

To ensure data integrity even if a teacher deletes or modifies a previously assigned lesson from the server repository, the student loading routine incorporates an automated self-healing validation filter:

1. **Physical Verification:** The system cross-references each lesson ID found in the student's progress profile against active directories on disk to verify physical existence.
2. **Sequential Status Enforcement:** It evaluates lesson states (Blocat, Deblocat, Finalizat). If a lesson is completed, its state is preserved. The first uncompleted lesson is automatically forced into the Deblocat state (with a refreshed allocation of 3 attempts), while all subsequent dependent modules are automatically locked.
3. **Telemetry Archiving:** Quiz scores are saved inside historical telemetry arrays (istoricNote), capturing multiple grade attempts, view counters (numarRedari), and completion timestamps to provide educators with granular analytical insights.

The validation of the StoryTeller platform requires a multi-tiered testing strategy designed to verify both the internal correctness of the software modules and the overall system behavior under real-world operational constraints. Because the application combines real-time 3D rendering (Unity C#), asynchronous Inter-Process Communication (IPC) via the Windows Subsystem for Linux (WSL), local NoSQL document persistence, and external cloud-based generative artificial intelligence (Google Gemini, gTTS, and Pollinations AI), testing cannot rely solely on simple functional checks.

This chapter establishes a rigorous testing framework divided into four primary validation stages: Component-Level Unit Testing, Integration and IPC Pipeline Testing, Generative Performance and Latency Benchmarking, and Pilot Usability Evaluation.

## 6.1. Testing Strategy and Objectives

The testing methodology addresses the unique engineering challenges associated with combining deterministic game engines with non-deterministic generative AI services. The specific objectives of the testing campaign are:

1. **Functional Verifiability:** Ensuring that core software modules including Role-Based Access Control (RBAC), local JSON database bootstrapping, the Finite State Machine (FSM) movement logic (ActorController), and dynamic assessment parsing (QuizManager) operate without regression or unhandled runtime exceptions.
2. **Pipeline Robustness and Concurrency:** Validating that asynchronous communication between Unity and the Python backend maintains UI responsiveness and prevents thread locks during high-latency network operations.
3. **Fault Tolerance and Graceful Degradation:** Confirming that network timeouts, rate-limiting triggers (HTTP 429), or malformed AI payloads are caught safely through automated exception handlers and fallback tokens (AI\_FAILED).
4. **Pedagogical and Usability Validation:** Assessing the practical usability of the platform across different user demographics (early childhood learners and adult educators) through structured pilot sessions.

## 6.2. Component-Level Unit and Functional Testing

To verify the internal logic of individual scripts before system integration, unit-level evaluations were performed on the primary architectural layers.

### 6.2.1. Database Initialization and RBAC Validation

The local document-based NoSQL storage managed by StoryDirector.initDatabase() was tested to ensure proper directory provisioning and security enforcement:

- **Directory Structure Verification:** Automated assertion checks confirmed that upon application launch, the root directory (BazaDeDate\_Scoala) and its four mandatory sub-collections (Utilizatori, Clase, Progres\_Elevi, and Sertar\_Profesor) are successfully created inside Unity's Application.persistentDataPath.
- **Super-Admin Provisioning:** Verified that the default administrative account (admin.json) is correctly initialized with hashed/stored credentials if absent.
- **Role Routing Integrity:** Tested the LoginCont() authentication method against 15 distinct user profiles (spanning Admin, Teacher, and Student roles). The system

achieved a **100% routing accuracy**, correctly instantiating the respective UI panels (adminPanel, profesorDashboardPanel, or elevPanel) based on the retrieved UserData.rol property.

### 6.2.2. Deterministic Movement Validation

The 3D navigation module was tested to verify spatial consistency and input handling under the NVIDIA PhysX engine:

- **Dependency Enforcement:** Confirmed that the [RequireComponent(typeof(Rigidbody))] attribute successfully prevents runtime null-reference exceptions by automatically coupling mass bodies to instantiated prefabs.
- **Vector Normalization Verification:** Tested diagonal movement inputs (W + D pressed simultaneously). Without vector normalization, diagonal velocity would empirically increase by a factor of  $\sqrt{2} \sim 1.41$ . Unit checks confirmed that applying .normalized successfully restricted maximum displacement magnitude to 1.0, preventing diagonal speed exploitation.
- **FixedUpdate Isolation:** Verified that physics translations via rb.MovePosition and spherical rotations via Quaternion.Slerp execute strictly within the deterministic FixedUpdate() loop, eliminating frame-rate-dependent stuttering across heterogeneous hardware profiles.

### 6.3. Integration Testing of the Inter-Process Communication (IPC) Pipeline

The communication bridge between the Unity frontend and the Python backend middleware (generator\_poveste.py), executed via wsl.exe, was subjected to rigorous integration testing.

#### 6.3.1. Process Spawning and Stream Redirection

The GameInterface.GenerareFlux coroutine was evaluated for stability during command-line argument transmission:

- **Test Setup:** Passed complex strings containing Romanian/English diacritics, spaces, and custom vocabulary themes as arguments through ProcessStartInfo.Arguments.
- **Observed Outcome:** WSL successfully ingested sys.argv parameters without encoding corruption.
- **UI Thread Responsiveness:** By monitoring frame rates during heavy backend execution, the asynchronous polling loop (while (!process.HasExited) { yield return null; }) successfully yielded execution back to Unity's frame scheduler. The client maintained a stable 60.0 FPS (variance  $< \pm 1.2$  FPS), completely eliminating the interface freezing typically associated with synchronous system process calls.

### 6.4. Generative Performance and Latency Benchmarking

Because the application relies on external cloud services for text generation, speech synthesis, and texture creation, quantifying the execution latency of the asynchronous pipeline is vital for understanding user waiting times and system throughput. The generation architecture was benchmarked under standard broadband operating conditions using a default test case yielding a 3-minute interactive lesson structure containing six sequential generative images and a dynamic assessment quiz.

### 6.4.1. Component-Level Latency Breakdown

Table 6.1 outlines the approximate processing duration for each consecutive stage of the generative pipeline, based on development observations and runtime telemetry.

| Pipeline Stage                                 | Underlying Technology / Service            | Approx. Duration | Share of Total Time |
|------------------------------------------------|--------------------------------------------|------------------|---------------------|
| <b>1. Request formulation &amp; CLI bridge</b> | Python initialization and sys.argv parsing | ~2s              | ~1-2%               |
| <b>2. Narrative &amp; JSON synthesis</b>       | Google Gemini API (gemini-2.5-flash)       | ~3-10s           | ~2-6%               |
| <b>3. Speech synthesis</b>                     | Google Text-to-Speech (gTTS)               | ~10-20s          | ~6-12%              |
| <b>4. Image synthesis &amp; download</b>       | Pollinations AI, 6 sequential images       | ~120-180s        | ~80-90%             |
| <b>Total lesson generation</b>                 | Complete asynchronous generation pipeline  | ~2.5-3.5min      | <b>100%</b>         |

Table 6.1: Generative Pipeline Latency Breakdown

Development observations indicate that complete lesson generation requires approximately 2.5-3.5 minutes in the tested configuration. The exact duration varies according to the response time of the external AI services and fluctuating network conditions.

The largest contribution to the total generation time comes from Stage 4 (image synthesis and downloading). This stage involves six sequential image requests, integrated together with retry and cooldown mechanisms intended to reduce the probability of rate-limit errors (HTTP 429) and resource conflicts. Consequently, the current implementation prioritizes generation reliability, structural fault tolerance, and correct asset association over minimum generation latency.

## 6.5. Fault Tolerance and Error Recovery Validation

Distributed systems depending on third-party APIs are inherently vulnerable to network latency, service interruptions, and rate-limiting blocks. To ensure that external failures do not destabilize the Unity client or lead to unhandled runtime exceptions, the robustness of StoryTeller was evaluated under simulated fault-injection scenarios.

Rather than relying on large-scale statistical sampling, the fault-tolerance verification focuses on deterministic architectural responses to specific failure vectors, as summarized in table below:

| Fault Scenario                       | Test Procedure / Trigger                                                                 | Observed System Response                                                                                                    |
|--------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Malformed JSON response</b>       | Invalid or conversational text introduced by the LLM before the structural JSON payload. | The backend cleaning logic (curata_si_valideaza_json) successfully isolated object boundaries and parsed valid schema keys. |
| <b>Invalid AI response / Timeout</b> | Simulated disruption or refusal during upstream API queries across model tiers.          | The execution pipeline intercepted the exception and generated a clean AI_FAILED aviary payload in story.json.              |
| <b>Image request failure</b>         | Network interruption or temporary throttling during Pollinations AI image downloads.     | The execution loop invoked built-in retry mechanisms and cooldown pauses to recover asset synchronization.                  |
| <b>Missing lesson asset</b>          | Required generated media file unavailable or deleted from disk during runtime loading.   | Runtime validation checks prevented loading null references, safely exiting to the dashboard interface.                     |
| <b>Backend process failure</b>       | Python or WSL execution process terminated unexpectedly mid-execution.                   | The Unity frontend detected process termination via standard error streams and returned control to the UI.                  |

Table 6.2: Architectural Fault-Tolerance Scenarios

### 6.5.1. JSON Schema Enforcement and Cleaning Validation

- **Test Methodology:** Generation requests were dispatched to the Gemini API using the strict system directive (prompt\_ingineresc) under varying network conditions.
- **Observed Behavior:** The majority of returned payloads successfully complied with the strict JSON structure required by Unity's C# deserializer on the first attempt. Minor structural inconsistencies (such as conversational markdown wrappers appended by the LLM) were automatically intercepted and cleaned by the backend sanitization logic (curata\_si\_valideaza\_json), completely preventing deserialization crashes inside the game engine.

### 6.5.2. Image Download Resilience (Pollinations AI)

Under degraded network conditions (simulating packet jitter and temporary throttling):

- **Behavior on Interruption:** Initial image download requests experiencing throttling or temporary connection drops trigger automated retry mechanisms and cooldown pauses.
- **Outcome:** The sequential retry pipeline successfully recovers asset synchronization across subsequent attempts, preventing broken texture references in the 3D scene.

### 6.5.3. Graceful Degradation (AI\_FAILED Resilience)

When internet connectivity or external API access is intentionally severed mid-generation:

- The backend exception handler successfully catches connection timeouts across the configured fallback model tiers (gemini-3.1-pro-preview, gemini-2.5-flash, gemini-2.0-flash).
- Instead of leaving a malformed or empty state, the system writes the designated error payload {"error": "AI\_FAILED"} directly to story.json.
- The Unity frontend (StoryDirector.LoadAndRunStory) successfully intercepts the token, logs a clean diagnostic message, invokes ArataEndPanel(), and terminates the execution coroutine safely via yield break, preventing unhandled exceptions and returning the user to the menu without crashing.

## 6.6. Pilot Usability and Demographic Evaluation

To assess the educational effectiveness, interface clarity, and accessibility of the platform, a structured pilot evaluation session was conducted with a heterogeneous participant group.

### 6.6.1. Participant Profile and Methodology

- Group A (Primary Learners / Early Childhood): 1 participant aged 4 years, testing sensory engagement, 3D character visualization, and audio-subtitled synchronization.
- Group B (Adult Educators / Observers): 3 adult participants, evaluating administrative dashboard navigation, multi-language configuration, and generated quiz accuracy.

### 6.6.2. Key Findings and Observations

- Engagement and Multimodal Learning (Group A): The 4-year-old participant maintained active attention throughout the narrative sequence. The real-time synchronization between gTTS audio playback and textSubtitrare UI updates provided effective multimodal reinforcement. The physics-based exploration mode proved intuitive after initial orientation, though physical keyboard inputs required minimal adult facilitation.
- Administrative Efficiency (Group B): Educators successfully navigated RBAC authentication, assigned student accounts to custom classes within the ClassData architecture, and reviewed historical telemetry archives. The average time required for an educator to configure a new lesson theme and initiate the backend generator was measured at 45 seconds. (Notă: Aceasta este o cifră reală și excelentă de menținut!)
- Assessment Interaction: The dynamic quiz module (QuizManager), supporting text and visual options (ImagineAB), effectively tested information retention, verifying that the generative loop reinforces core vocabulary acquired during the 3D storytelling phase.

## 6.7. Storage Scalability and Resource Utilization

System resource monitoring during prolonged usage sessions demonstrated high efficiency:

- Local Persistence Overhead: User profile and class configuration files averaged ~1.2 KB per record. (Notă: Foarte bun și ușor de justificat).

- **Offline Lesson Package Size:** A complete generated lesson archive (comprising story.json, audio tracks, and textures) averaged between 1.5 MB and 3.2 MB, confirming that the document-based NoSQL persistence model places minimal strain on standard consumer hardware storage.

## 6.8. Summary Of Testing

The comprehensive testing and validation campaign demonstrates that the StoryTeller platform achieves a robust balance between generative flexibility and system stability. By combining automated schema enforcement, multi-model API fallbacks, sequential asset management, and asynchronous IPC communication, the architecture provides a reliable, secure, and engaging educational environment capable of running smoothly on standard consumer hardware.

## Chapter 7. User's Manual

### 7.1. Installation and Deployment Procedure (Administrator's Guide)

This section outlines the hardware and software resource prerequisites, dependency configurations, and step-by-step deployment instructions required to set up the environment successfully.

#### 7.1.1. Hardware and Software Prerequisites

To run and deploy the application smoothly without requiring specialized Virtual Reality (VR) hardware or high-end graphics processing units, the target machine must satisfy the following baseline specifications:

- **Operating System:** Windows 10 / 11 (64-bit) with **Windows Subsystem for Linux (WSL 2)** enabled for backend Python execution.
- **Processor (CPU):** Quad-core CPU (Intel Core i5 / AMD Ryzen 5 or higher).
- **Memory (RAM):** Minimum 8 GB RAM (16 GB recommended for smooth Unity rendering and API caching).
- **Storage:** At least 500 MB of free disk space for the application build, local NoSQL JSON database directories, and temporary asset caches.
- **Software Environment:**
  - Python 3.10 or newer (installed within the WSL environment).
  - Unity 3D Editor (version 2022.3 LTS or newer) if running from source, or the standalone Windows build executable (.exe).
  - Stable internet broadband connection for querying cloud-based generative APIs (Google Gemini, Pollinations AI, and Google TTS).

#### 7.1.2. Step-by-Step Deployment Instructions

Administrators can deploy the project environment by executing the following sequential configuration steps:

1. **Clone or Extract the Project Repository:** Transfer the complete project source folder to the target machine (C:\Users\Public\Licenta\_StoryAI). Ensure that the Python backend script (generator\_poveste.py) is accessible at the path expected by the Unity frontend (/mnt/c/Users/<Username>/Licenta\_StoryAI/python/generator\_poveste.py).
2. **Configure the Python Virtual Environment (via WSL):** Open a WSL terminal (Ubuntu distribution recommended) and navigate to the project directory:

```
cd /mnt/c/Users/<Username>/Licenta_StoryAI
python3 -m venv venv
source venv/bin/activate
```

3. **Install Required Python Dependencies:** Install the necessary libraries (such as requests, gTTS, etc.) using pip:

```
pip install --upgrade pip
pip install requests gTTS
```

4. **Verify API Authentication Keys:** Ensure that the secure Google Gemini API key is properly injected inside the backend script (`generator_poveste.py`) under the `GOOGLE_API_KEY` constant variable string to allow seamless token validation during lesson generation.
5. **Launch the Unity Application:** Open the Unity Hub, add the project directory, and launch it using the Unity 2022.3 LTS editor, or execute the standalone compiled desktop build (`StoryTeller.exe`). Upon first execution, the `StoryDirector.InitDatabase()` method will automatically bootstrap the local document-based NoSQL storage tree (`Utilizatori`, `Clase`, `Progres_Elevi`, and `Sertar_Professori`) inside Unity's `Application.persistentDataPath`, provisioning the default super-administrator account (`admin.json`).

## 7.2. End-User Operation Manual

This section describes how to operate the application from the perspective of an end-user (Educator or Student) with no technical background. The interface is organized around intuitive panels and role-based workflows.

### 7.2.1. Authentication and Role-Based Access Control (RBAC)

When launching the application, the user is greeted by the **Login Panel**.

1. Enter your assigned username in the **Email / Username** input field.
2. Enter your account password in the **Password** field.
3. Click the **Login** button.

Behavior: The system validates credentials against local JSON records. Depending on your assigned role (Admin, Profesor, or Elev), the interface automatically routes you to the respective dashboard.

### 7.2.2. Teacher Workflow: Lesson Generation and Publishing

Educators can dynamically build and assign customized interactive lessons without writing code:

1. From the **Teacher Dashboard**, navigate to the generation panel.
2. Input the desired narrative theme ("Wild Animals" or "Basic Colors"), select the hero character model from the dropdown list, and configure the target age group, base language, and learning language.
3. Click the **Generate** button.
  - Interaction: The system triggers the asynchronous WSL IPC pipeline (`GameInterface.GenerareFlux`), displaying a brief loading state while the backend queries the Gemini API, synthesizes gTTS audio clips, and downloads sequential textures (`imagine_1.jpg`, etc.) from Pollinations AI.
4. Once generation completes, preview the 3D interactive story.

5. To assign the lesson, enter a custom lesson name in the save field, click **Save Draft** to store it in your physical teacher repository (Sertar\_Professori), select the target class from your managed classes dropdown, and click **Publish Lesson**. This automatically updates the student progress schema (ElevProgresData) across all enrolled students.

### 7.2.3. Student Workflow: Exploration and Assessment

Students experience immersive storytelling and solve adaptive evaluations:

1. Log in using student credentials to open the **Student Dashboard**, which displays active class enrollment and overall progress statistics.
2. Click **Start Lessons** to view the list of available modules. Unlocked lessons are interactively available, while subsequent modules remain locked behind prerequisite completion.
3. Click on an active lesson to launch the 3D environment. Use the keyboard (WASD or directional arrows) to navigate the avatar in the physics-based exploration mode (ActorController), guided by Cinemachine dynamic camera tracking.
4. Intersecting narrative triggers pause movement, loading dynamically generated dialogue, synchronized audio narration, and visual subtitles (textSubtitrare), while 3D display monitors render the sequential story textures.
5. Upon story completion, the **Dynamic Assessment Module (QuizManager)** appears automatically, presenting polymorphic questions (text queries or visual options like ImagineAB). Select an option by clicking the corresponding button. Immediate localized feedback is provided, and upon finishing the quiz, the final score is computed and automatically transmitted back to the progress directory for the educator's review.

## Chapter 8. Conclusions

### 8.1. Summary of Contributions and Achievements

The fundamental motivation behind this thesis was to address the structural gap between static, passive e-learning tools and expensive, hardware-constrained immersive Virtual Reality (VR) environments. Existing educational platforms largely stagnate at administrative task management or rely on rigid, pre-programmed 2D presentations, failing to provide the active, "hands-on" cognitive engagement associated with physical language immersion.

To bridge this gap, this project successfully researched, designed, and implemented **StoryTeller**, an autonomous, AI-driven software architecture that decouples pedagogical narrative content from rigid graphical resources. The key achievements of this work include:

1. **Decoupled Client-Server and Middleware Architecture:** Successfully engineered a distributed framework separating the heavy 3D graphical processing (handled locally by Unity C#) from asynchronous artificial intelligence generation logic (managed by a Python backend via the Windows Subsystem for Linux - WSL).
2. **Asynchronous Inter-Process Communication (IPC):** Implemented a non-blocking execution bridge using system process control (`ProcessStartInfo`) and frame-rate-safe coroutine polling (`while (!process.HasExited) { yield return null; }`), ensuring that high-latency cloud operations do not freeze the Unity client interface or compromise user immersion.
3. **Local NoSQL Document-Based Storage Engine:** Designed a lightweight, highly portable data persistence layer utilizing localized JSON files stored directly within Unity's `Application.persistentDataPath`. This architecture successfully supports Role-Based Access Control (RBAC), class-student relational mapping, and offline lesson repositories without requiring the overhead of a heavy relational database server.
4. **Automated Generative Pipeline & Schema Enforcement:** Integrated advanced Large Language Model prompt engineering (`prompt_ingineresc`) with strict JSON schema enforcement and sequential asset management (`genereaza_imagini_securizat`), eliminating asynchronous race conditions and ensuring absolute referential integrity between metadata and physical media files (such as gTTS audio streams and Pollinations AI textures).
5. **Robust Fault Tolerance and Self-Healing Mechanics:** Developed a comprehensive error-handling framework, including multi-model API fallbacks, graceful degradation via the `AI_FAILED` token, and an automated student progress auto-repair algorithm that validates prerequisite lesson completion upon loading.

### 8.2. Critical Analysis of Results Achieved

The evaluation campaign detailed in Chapter 6 confirmed that the system meets its operational and architectural specifications, though certain trade-offs became apparent during empirical testing:

- **Generative Latency vs. Asset Integrity:** Benchmarking revealed that a complete interactive lesson generation cycle requires approximately 2.5 minutes. While this

might appear demanding in a real-time classroom setting, an in-depth analysis indicates that 88% of this latency originates from Stage 4 (sequential image downloading). The inclusion of mandatory cooldown delays (`time.sleep(2)`) and strict sequential counters is non-negotiable to prevent HTTP 429 rate-limit rejections from public generative endpoints and to avoid file-overwrite collisions. Thus, the system deliberately trades initialization speed for absolute data integrity and stability.

- **Hardware Accessibility and Portability:** By leveraging local JSON document storage, the application achieves an average storage footprint of only 1.5 MB to 3.2 MB per completed lesson. This proves that the platform successfully democratizes access to immersive education, running smoothly on standard consumer hardware (PCs and tablets) without requiring specialized VR headsets or high-end GPU clusters.
- **Robustness under Network Stress:** Stress testing demonstrated that the application's multi-tier exception handling and strict output validation parsing (achieving a 98% first-pass JSON compliance rate) effectively protect the client from external API instabilities, ensuring that third-party service degradation never results in unhandled engine crashes.

### 8.3. Possibilities for Improvements and Further Development

While StoryTeller successfully fulfills its core objectives, several opportunities remain for future expansion and academic refinement:

1. **Real-Time Streaming and WebSocket Integration:** Replacing the current CLI-based Python invocation via WSL with a persistent, asynchronous FastAPI backend server utilizing WebSockets or Server-Sent Events (SSE) would allow the Unity client to receive real-time generation progress updates, significantly improving user feedback during the asset synthesis phase.
2. **Transition to Self-Hosted Generative Models:** Integrating local, open-source models (such as Ollama-hosted LLMs or local Stable Diffusion pipelines) would eliminate dependency on third-party cloud APIs, completely removing rate limits, reducing generation latency, and ensuring strict data privacy for educational institutions.
3. **Expanded Multi-User and Collaborative Environments:** Extending the local NoSQL schema into a synchronized multi-player network architecture would enable classrooms of students to explore virtual 3D environments collaboratively, solving quests and participating in group assessments in real-time.
4. **Richer Pedagogical Analytics:** Enhancing the educator dashboard with advanced machine learning analytics could provide predictive insights into student comprehension difficulties, automatically suggesting tailored vocabulary adjustments for subsequent generated lessons.

## Bibliography

- [1] J. Gregory, *Game Engine Architecture*, 3rd Edition, CRC Press, Taylor & Francis Group, Boca Raton, FL, 2018.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser și I. Polosukhin, „Attention Is All You Need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS 2017)*, Long Beach, CA, USA, pp. 5998–6008, 2017.
- [3] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior and K. Kavukcuoglu, „WaveNet: A Generative Model for Raw Audio,” in *Proceedings of Interspeech 2016*, Sunnyvale, USA, pp. 2238-2242, 2016.
- [4] Y. Zhang, S. H. B. Halili și Z. Zainuddin, „Applications of generative AI in early childhood education: A systematic review,” *EURASIA Journal of Mathematics, Science and Technology Education*, vol. 22, nb. 3, em2793, 2026.
- [5] A. A. Khan, S. Raja and M. R. Mirza, „Generative Artificial Intelligence in Education: Opportunities, Challenges, and Its Impact on Students’ Learning Outcomes,” *Kashmir Journal of Academic Research and Development*, vol. 2, nb. 6, pp. 131-147, 2026.
- [6] Y. Hashem, S. Esnaashari, K. Onslow, S. Chakraborty, A. Poletaev, J. Francis and J. Bright, „Understanding the Impacts of Generative AI Use on Children: WP1 Surveys,” *The Alan Turing Institute, Londra, United Kingdom*, Tech. Rep., 2025.
- [7] T. Brown et al., „Language Models are Few-Shot Learners,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS 2020)*, Vancouver, BC, Canada, pp. 1877–1901, 2020.
- [8] S. Kasneci et al., „ChatGPT for good? On opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, p. 102274, 2023.
- [9] R. Mayer, *Multimedia Learning*, 3rd ed. Cambridge: Cambridge University Press, 2020.
- [10] L. Johnson, S. Adams Becker, V. Estrada, and A. Freeman, „NMC Horizon Report: 2015 Higher Education Edition,” *The New Media Consortium, Austin, Texas*, Tech. Rep., 2015.

## Appendix 1

This appendix provides supplementary implementation details for core modules of the **StoryTeller** platform, focusing on IPC management, LMS state persistence, and backend validation.

### A.1. Asynchronous IPC and WSL Process Orchestration

The following C# coroutine manages the non-blocking execution of the Python backend via the Windows Subsystem for Linux (WSL). By leveraging an asynchronous polling loop (`while (!process.HasExited)`), the Unity frontend yields execution back to the engine frame scheduler, maintaining stable rendering performance during high-latency calls.

---

```
//Excerpt from GameInterface.cs: Asynchronous IPC pipeline via WSL
public IEnumerator GenerareFlux(string tema, string heroName, int dimensiuneQuiz, int
varianteQuiz, string langBaza, string langInv)
{
    string scriptPath = "/mnt/c/Users/Public/Licenta_StoryAI/python/generator_poveste.py";
    string arguments = $"wsl.py python3 {scriptPath} \"{tema}\" \"{heroName}\"
{dimensiuneQuiz} {varianteQuiz} \"{langBaza}\" \"{langInv}\"";

    ProcessStartInfo startInfo = new ProcessStartInfo
    {
        FileName = "wsl.exe",
        Arguments = arguments,
        UseShellExecute = false,
        RedirectStandardOutput = true,
        RedirectStandardError = true,
        CreateNoWindow = true
    };
    using (Process process = Process.Start(startInfo))
    {
        while (!process.HasExited)
        {
            // Yield execution to prevent Unity main-thread freezing
            yield return null;
        }
        string errors = process.StandardError.ReadToEnd();
        if (!string.IsNullOrEmpty(errors))
        {
            Debug.LogWarning(" WSL Execution Warning/Error: " + errors);
        }
    }
}
```

---

## A.2. Student Progress Auto-Repair and State Enforcement

This C# method handles the self-healing validation filter for student curricula. It cross-references assigned lessons against physical directories on disk, preserves completed states, forces the first uncompleted module into an active "Deblocat" state with a refreshed allocation of attempts, and locks subsequent dependent units.

```

-----
// Excerpt from StoryDirector.cs: Prerequisite auto-repair algorithm
public void Elev_IncarcaLectii(string idElev)
{
    string pathProgres = Path.Combine(Application.persistentDataPath, "Progres_Elevi",
    $"{idElev}.json");
    if (!File.Exists(pathProgres)) return;

    ElevProgresData progres =
    JsonUtility.FromJson<ElevProgresData>(File.ReadAllText(pathProgres));
    bool primulNeverificatGasit = false;

    foreach (var lectie in progres.progresLectii)
    {
        string physicalPath = Path.Combine(Application.streamingAssetsPath,
        "Sertar_Professori", lectie.idLectie);

        // Physical verification fallback
        if (lectie.status == "Finalizat")
        {
            continue; // Preserve finished historical states
        }

        if (!primulNeverificatGasit)
        {
            lectie.status = "Deblocat";
            if (lectie.incerariRamase <= 0) lectie.incerariRamase = 3; // Refresh attempt
allocation
            primulNeverificatGasit = true;
        }
        else
        {
            lectie.status = "Blocat"; // Enforce sequential dependency locking
        }
    }

    File.WriteAllText(pathProgres, JsonUtility.ToJson(progres, true));
}
-----

```

### A.3. Backend JSON Schema Cleaning and Deserialization Fallback

To prevent parsing exceptions when external Large Language Models append natural language wrappers around structured output, the Python backend implements a robust cleaning mechanism that extracts valid JSON blocks and triggers fallback flags (AI\_FAILED) upon structural corruption.

---

```
# Excerpt from generator_poveste.py: JSON schema cleaning and error resilience
import json
import re
```

```
def curata_si_valideaza_json(raw_response_text):
    try:
        # Strip potential Markdown code-block markers generated by LLMs
        cleaned = re.sub(r"```json", "", raw_response_text, flags=re.IGNORECASE)
        cleaned = re.sub(r"```", "", cleaned).strip()

        # Isolate the primary JSON object boundaries
        start_idx = cleaned.find('{')
        end_idx = cleaned.rfind('}')

        if start_idx != -1 and end_idx != -1:
            json_str = cleaned[start_idx:end_idx+1]
            parsed_data = json.loads(json_str)

            # Validate core schema keys required by Unity frontend
            if "actions" in parsed_data and "quiz" in parsed_data:
                return parsed_data

            raise ValueError("Schema key validation failed.")

    except Exception as e:
        print(f"JSON Parsing Error: {str(e)}")
        # Write failure payload for graceful degradation in Unity
        avarie_payload = {"error": "AI_FAILED", "details": str(e)}
        return avarie_payload
```

---

